

Design of Production Agentic Systems

Lecture 2: Agent architectures and execution machinery

Rahul Krishna

Columbia University, COMS E6998-019, Fall 2026

These notes cover the first fifty-minute block of the Week 2 meeting: agent architectures. We build one patch-generation agent, mini-swe-agent, through five preserved versions, from a fixed workflow to a reflective attempt loop, and we ask the same question of each version: which decisions did we make at design time, and which are deferred to the model at run time? The vocabulary is AIMA's; the mechanisms are those of the assigned papers, quoted from the versions each section's reading list names. Every section opens with the numbered claims it defends and closes with exercises and the reading it rests on. Listings and traces that depend on demo code not yet built are marked as stubs naming the artifact expected there. Part B turns to the machinery that carries a decision from one step to the next: the record the runtime keeps, the code that reads it to choose the next node or to stop, and the boundary at which a requested action becomes an executed one.

Date: Draft of September 16, 2026, for the meeting of September 18, 2026

Contents

Recap	3
1 Intelligent agents	3
1.1 Rational agents	3
1.1.1 The running example: mini-swe-agent	4
1.1.2 Is this a rational agent?	5
1.2 The task environment	6
1.3 Properties of task environments	6
1.3.1 Fully observable vs. partially observable	6
1.3.2 Single-agent vs. multiagent	6
1.3.3 Deterministic vs. nondeterministic	7
1.3.4 Episodic vs. sequential	7
1.3.5 Static vs. dynamic	7
1.3.6 Discrete vs. continuous	7
1.3.7 Known vs. unknown	7
1.4 Where the vocabulary lives in mini-swe-agent	7
1.5 Exercises	8
1.6 Reading	8
2 Fixed workflows	9
2.1 Control decisions and content decisions	9
2.2 What we gain and what we lose	10
2.2.1 What we gain	10
2.2.2 What we lose	10
2.3 Agentless, one realization	10
2.4 Exercises	12
2.5 Reading	12
3 Reactive loops	13
3.1 The action that only changes the context	13
3.2 What we gain and what we lose	14
3.2.1 What we gain	14
3.2.2 What we lose	14
3.3 How a reactive loop fails	14
3.4 The demo: the loop	15
4 Planning, validation and reflection	17
4.1 The plan as a field of state	17
4.2 Three shapes of planner	18

4.3	Revising plans	19
4.4	Does planning make a difference?	19
4.5	The proposer and the judge	20
4.6	What a model judge can and cannot see	21
4.7	Why the judge does not decide acceptance	21
4.8	What survives a refusal	22
4.9	What we gain and what we lose	22
4.9.1	What we gain	23
4.9.2	What we lose	23
4.10	The demo: the planner and the validator	23
4.11	Exercises	24
4.12	Reading	24
5	Other agent design patterns	25
5.1	The problem the reading addresses	25
5.2	Mechanism	25
5.3	Evidence	25
5.4	Which decisions the programmer made	27
5.5	The demo	27
5.6	Exercises	28
5.7	Reading	28
6	The investigation record	29
6.1	What connects one step to the next	29
6.2	Model context, runtime state, durable state	29
6.3	The record is factored	30
6.4	Reducers, and what a later step is allowed to see	31
6.5	The demo: one tool result becoming state	31
6.6	Exercises	32
6.7	Reading	32
7	Routing, feedback loops, and termination	32
7.1	Routing is code, not prose	32
7.2	The same graph, different paths	33
7.3	Two returns to the planner	33
7.4	What actually stops a loop	33
7.5	What a trace establishes	34
7.6	The demo: three endings	34
7.7	Exercises	34
7.8	Reading	35
8	The tool boundary	35
8.1	The tool contract	35
8.2	The dispatch cycle	35
8.3	What the model saw, and what ran	36
8.4	The interface is a design decision	36
8.5	Execution authority	37
8.6	The demo: a refused request	37
8.7	Exercises	38
8.8	Reading	38

Part A: Agent architectures

Recap

Last week we drew an agentic system as five component roles and one caller, and gave every box an owner.

- The *caller* sits outside the agent. It starts or resumes a run, supplies the task, the input, and any constraints, and receives the result.
- The *model* interprets observations and selects the next action: it reasons, plans, and proposes.
- The *runtime* owns control flow, validation, scheduling, and termination. It executes the loop, keeps the budget, and writes the terminal event.
- *Tools* are the only path from a proposal to a real effect: narrow, typed interactions with external systems.
- *State* is what outlives the call, the step, and the process. *Context* is what the model can see on this call.
- The *environment* holds external truth: the sources of data the agent observes and can change, and the failures it did not cause.

Two sentences carry these notes. The caller starts the run, the runtime executes the loop, and the model proposes what to do next. A requested action is not an executed action, and an executed action is not a verified outcome.

We also keep three habits. Token usage is four counters, input, output, cache_read, and cache_create, and is never summed. Three terminal statuses exist, ok, error, and no terminal event at all, and the third is never folded into the second. Every number about a published system is read from the version cited and pointed at its table or figure.

1 Intelligent agents

The upshot

1. **The agent** is a mapping from percept sequences to actions. The agent function is that mapping; the agent program is the code that computes it, and the program is where our design choices live. `mini-swe-agent` is the running example: its percepts are one alert and whatever its tools return, and its actions are `read_file`, `search`, `edit`, and `submit`.
2. **The environment** is specified before the program is designed, as a PEAS description, and classified by seven properties that decide which programs can work. Ours is partially observable, single-agent, deterministic, sequential within one alert, static, discrete, and known. One property matters: partially observable means a rational agent here must act to see.
3. **The performance measure** says which sequences of environment states count as good. It is ours, it scores the environment and not the agent's beliefs, and rationality is defined against it: select the action expected to do best given the percept sequence so far. For `mini-swe-agent`, a patch that removes the reported weakness and leaves the endpoint behaving as it did, at bounded cost.
4. **One question orders everything that follows:** how much of the percept-to-action mapping do we fix at design time, and how much does the program compute from percepts at run time? A fixed workflow fixes nearly all of it and is rational only while the designer's prior knowledge holds. A reactive loop computes the next action from the percepts so far. A planner, a validator, and a reflection step each move one more decision to run time, and the design patterns compose these. `mini-swe-agent` walks that order, one version per step.

1.1 Rational agents

Definition 1.1 (Rational agent). “For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has” [14, §2.2.2].

The definition uses three terms we fix first. An agent is anything that perceives its environment through sensors and acts on it through actuators [14, §2.1]; a software agent's sensors deliver file contents and network input, and its actuators write files and send output. A *percept* is the content the sensors deliver at one instant, and the *percept sequence* is the complete history of everything the agent has perceived. The agent's behavior is described by its *agent function*, which maps any percept sequence to an action, and an agent's choice may depend on that sequence and its built-in knowledge but on nothing it has not perceived [14, §2.1]. A *performance measure* evaluates the sequence of environment states the agent's actions bring about [14, §2.2.1]: it scores the environment, not the agent's beliefs about it. One more distinction carries the rest of these notes. The agent function is an abstract description of behavior, while the *agent program* is the concrete implementation that computes it, running on an architecture that makes percepts available and carries out the actions the program chooses [14, §2.1, §2.4]. We fix the function when we specify the task, and the program is where our design choices live.

What is rational at any given time depends on four things [14, §2.2.2]:

- the performance measure that defines the criterion of success;
- the agent's prior knowledge of the environment;
- the actions that the agent can perform;
- the agent's percept sequence to date.

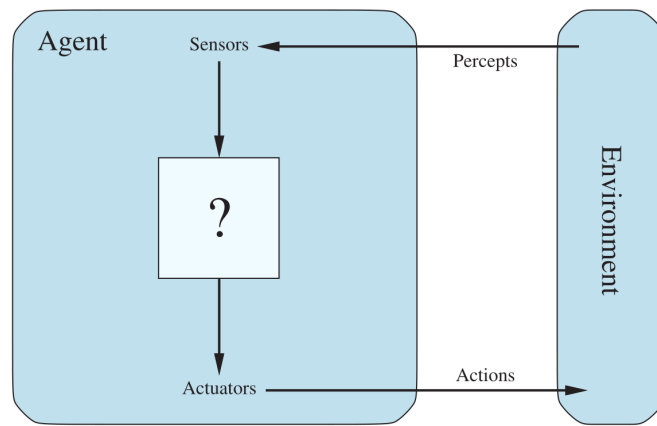


Figure 1 An agent and its environment. Figure 2.1 of Russell and Norvig [14], reproduced from the Global Edition (© Pearson Education Limited 2022).

1.1.1 The running example: mini-swe-agent

mini-swe-agent is the agent we build through five versions in this block. Its environment is one application, OWASP BenchmarkPython at a pinned commit [13], together with the alerts a static scanner reports against it. It takes one alert at a time, reads the alert as an issue report, localizes the code the alert names, edits that code, and submits a patch. Figure 2 is the whole agent: three states and five transitions, with each label read as an event and then a condition in brackets.

- **Controller** decides the next action.
- **Tool** executes the action the controller requested and hands back an observation. There is one node per tool, and this version has three: `read_file` over a range of lines, `search` for a pattern, and `edit` as a replacement in one file that the runtime applies to the working tree.
- **Submit** validates the submission. On *accepted* the run ends. On *rejected* the feedback goes back to the controller, which may act again.

Every version in this block is a change to that machine: a state, an edge, or a different component deciding. Nothing else about the agent changes.

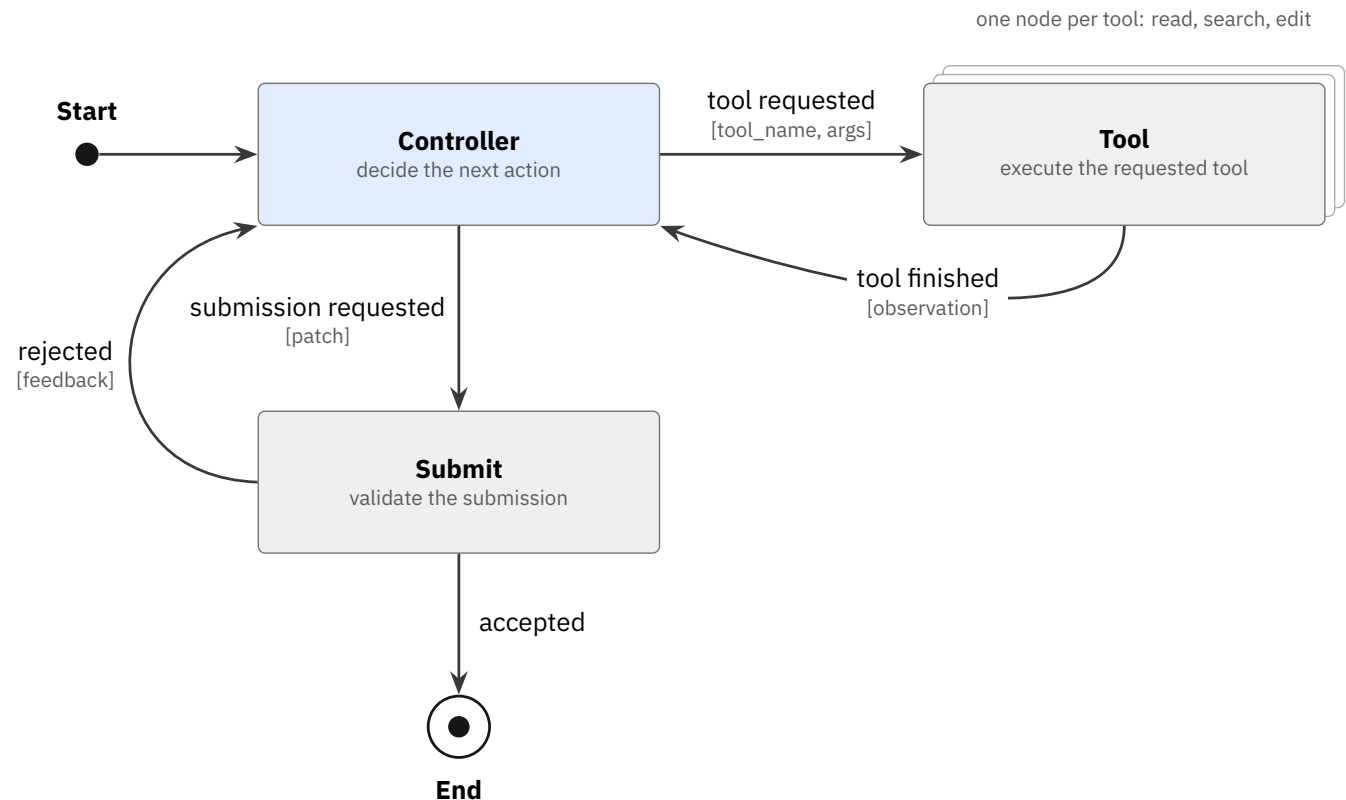


Figure 2 mini-swe-agent as a state machine. Blue is the state where a model call decides, grey is runtime-owned. The run ends only when a submission is accepted.

One alert serves as the worked example throughout: Bandit's rule B608, string-based SQL query construction, at line 46 of `testcode/BenchmarkTest00283.py`. Line 46 builds the query by interpolating the request value `bar` into the string, and

line 49 executes it. The patch is two lines in one file, a placeholder in the query and bar bound when the query runs.

Listing 1 The worked alert, before and after. Two lines in one file.

```
# before: the value is interpolated into the query text
sql = f'SELECT username from USERS where password = \'{bar}\''
cur.execute(sql)

# after: the query carries a placeholder and the value is bound
sql = 'SELECT username from USERS where password = ?'
cur.execute(sql, (bar,))
```

One very simple agent function is the following. If the alert names a location that is not yet in the percept sequence, read it. If the code has been read but not edited, edit it. Once the edit is applied, submit. [Figure 3](#) traces that function on the worked alert. A full tabulation of it would be unbounded unless the budget bounds the percept sequence.

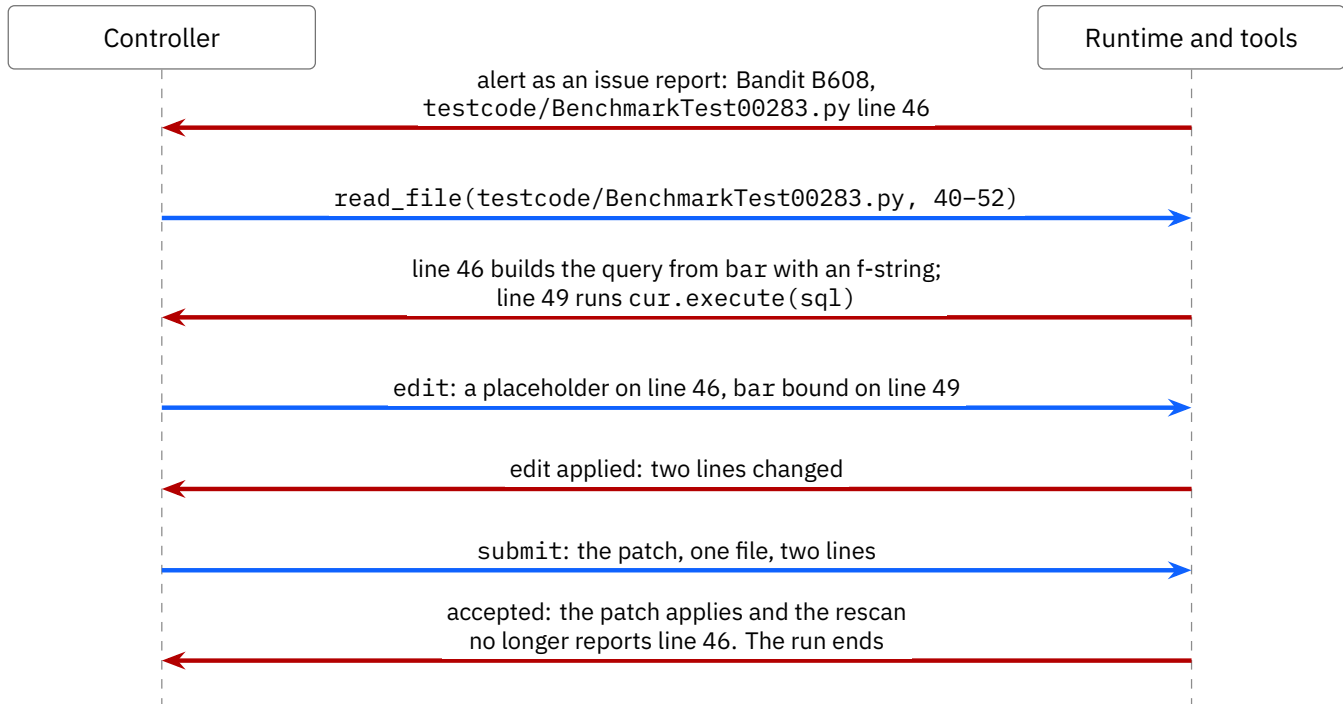


Figure 3 One run on the worked alert. Blue arrows are actions the controller requests; red arrows are percepts the runtime returns.

search does not appear in that run, because the alert names the line and the value it interpolates is assigned in the same function. It earns its place as soon as the fix has to touch code the alert does not name, which is the case [Section 1.1.2](#) turns on.

1.1.2 Is this a rational agent?

That depends. We have to say what the performance measure is, what is known about the environment, and what the agent can do and perceive. Assume the following.

- The performance measure awards one point if the submitted patch removes the weakness the alert reports and leaves the endpoint behaving as it did, over a lifetime of one alert.
- The geography of the environment is known a priori: the application's layout and the behavior of the tools. A read returns the lines of the working tree, a search returns every location that matches, and an edit replaces text in one file. Which change removes the weakness is not known.
- The only available actions are `read_file`, `search`, `edit`, and `submit`.
- The agent correctly perceives what its tools return.

Under these circumstances the agent is rational: reading the flagged lines before editing them is the action that maximizes expected performance, and no other agent does better in expectation.

The measure and the check are not the same thing, and the gap is worth naming now. The Submit state can establish that the patch applies, that it touches the line the alert named, and that a rescan no longer reports it. None of that establishes that the endpoint still returns what it returned before. Accepted is not correct, and [Section 4.5](#) is where that distinction becomes a component.

The same agent is irrational under different circumstances. If the performance measure charges a point for each tool call, an agent that reads whole files fares poorly, and a better agent reads the smallest range that answers its question. If the value the query interpolates arrives from a helper in another file, an agent that edits only the flagged line can leave the weakness in

place or change what the endpoint returns. A better agent searches for the helper first, and how far to follow the value is the depth of investigation we fix. And once the edit is applied and accepted, an agent that keeps editing wastes every further call and risks breaking behavior; a better agent submits as soon as it is sure. Those three circumstances are the budget, the depth of investigation, and termination, and each later section makes one of them a decision.

Food for thought for HW1

What are your yardsticks for the agent you are developing, and why? Before you write a node, write down its performance measure, the depth of investigation you will allow, and the rule that ends an investigation, and say for each what it rewards and what it lets through.

Three consequences of the definition matter for what follows [14, §2.2.3]. Rationality is not omniscience: it maximizes expected performance, not actual performance, and the agent cannot know whether its edit removes the weakness before it has read the code around the alert. Acting to change future percepts, information gathering, is part of rationality and not a detour: our agent reads the flagged function before it edits because looking is how it maximizes expected performance. And an agent that relies on its designer's prior knowledge rather than on its own percepts lacks autonomy. A fixed workflow, whose sequence of read, edit, and submit steps we write in advance, is the extreme of that dependence, and it is where Section 2 starts.

1.2 The task environment

Definition 1.2 (Task environment and PEAS). The performance measure, the environment, the actuators, and the sensors together form the *task environment*, written as a PEAS description. “In designing an agent, the first step must always be to specify the task environment as fully as possible” [14, §2.3.1].

Task environments are the problems to which rational agents are the solutions, and the nature of the task environment decides the appropriate design for the agent program [14, §2.3]. Table 1 is the PEAS description of mini-swe-agent.

Table 1 PEAS description of the task environment for mini-swe-agent.

Performance measure	Environment	Actuators	Sensors
The weakness the alert reports is gone and the endpoint still returns what it returned before; bounded cost in tool calls and tokens	OWASP BenchmarkPython at a pinned commit; one alert from a static scanner, read as an issue report; the working tree the agent edits	read_file(path, range), search(pattern), edit(path, old, new), submit(patch)	The alert; file excerpts; search hits; edit confirmations; the validation result; tool errors

The environment is not the whole world but the part whose state we care about when we design the agent: what affects what it perceives, and what its actions affect [14, §2.1]. Nothing outside the working tree and the alert affects what our agent perceives, and its actions affect nothing outside the working tree. The last column is the narrow part: every percept after the first arrives as the result of a tool call.

1.3 Properties of task environments

Task environments vary along seven dimensions, and the dimensions decide, to a large extent, which agent designs and which families of techniques apply [14, §2.3.2]. Each table below gives one pair: the defining sentence and the standard examples [14, Fig. 2.6].

1.3.1 Fully observable vs. partially observable

	Fully observable	Partially observable
Definition	“If an agent’s sensors give it access to the complete state of the environment at each point in time, then we say the task environment is fully observable.” Effectively fully observable if the sensors detect every aspect relevant to the choice of action.	Sensors are noisy or inaccurate, or parts of the state are missing from the sensor data. With no sensors at all, unobservable.
Examples	Crossword, chess with a clock, backgammon, image analysis	Poker, taxi driving, medical diagnosis, part-picking robot, refinery controller

1.3.2 Single-agent vs. multiagent

	Single-agent	Multiagent
Definition	No other entity’s behavior is best described as maximizing a performance measure whose value depends on this agent’s behavior.	“The key distinction is whether B’s behavior is best described as maximizing a performance measure whose value depends on agent A’s behavior.” Competitive (chess) or partially cooperative (taxi driving).
Examples	Crossword, medical diagnosis, image analysis, part-picking robot, refinery controller	Chess with a clock, poker, backgammon, taxi driving, English tutor

1.3.3 Deterministic vs. nondeterministic

	Deterministic	Nondeterministic
Definition	“If the next state of the environment is completely determined by the current state and the action executed by the agent(s), then we say the environment is deterministic.”	Otherwise. A partially observable environment can appear nondeterministic. <i>Stochastic</i> is reserved for models that quantify the possibilities with probabilities.
Examples	Crossword, chess with a clock, image analysis	Poker, backgammon, taxi driving, medical diagnosis, refinery controller (listed as stochastic)

1.3.4 Episodic vs. sequential

	Episodic	Sequential
Definition	“In an episodic task environment, the agent’s experience is divided into atomic episodes. In each episode the agent receives a percept and then performs a single action. Crucially, the next episode does not depend on the actions taken in previous episodes.”	“In sequential environments, on the other hand, the current decision could affect all future decisions.”
Examples	Image analysis, part-picking robot	Crossword, chess, poker, backgammon, taxi driving, medical diagnosis, refinery controller, English tutor

1.3.5 Static vs. dynamic

	Static	Dynamic
Definition	The environment cannot change while the agent is deliberating. The agent need not keep looking at the world while deciding, nor worry about the passage of time.	“If the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent.” <i>Semidynamic</i> if the environment does not change with time but the performance score does.
Examples	Crossword, poker, backgammon; chess with a clock and image analysis are semidynamic	Taxi driving, medical diagnosis, part-picking robot, refinery controller, English tutor

1.3.6 Discrete vs. continuous

	Discrete	Continuous
Definition	“The discrete/continuous distinction applies to the state of the environment, to the way time is handled, and to the percepts and actions of the agent.” A finite number of distinct states, percepts, and actions.	State, time, percepts, or actions sweep through a range of values.
Examples	Crossword, chess with a clock, poker, backgammon, English tutor	Taxi driving, medical diagnosis, image analysis, part-picking robot, refinery controller

1.3.7 Known vs. unknown

	Known	Unknown
Definition	“In a known environment, the outcomes (or outcome probabilities if the environment is nondeterministic) for all actions are given.” The distinction concerns the agent’s knowledge of the laws of the environment, not the environment itself.	The agent has to learn how the environment works before it can decide well. A new video game may show the whole screen and still not say what the buttons do.
Examples	Chess and poker: the rules can be supplied in full	Driving a rented car in a new country

The hardest case [14, §2.3.2] is partially observable, multiagent, nondeterministic, sequential, dynamic, continuous, and unknown. The task environment of *mini-swe-agent* is partially observable, single-agent, deterministic, sequential within one alert and episodic across alerts, static, discrete, and known: it shares two properties with the hardest case, partial observability and sequentiality, and none of the other five. Partial observability alone is what makes the task an investigation, and it is the property every later section responds to.

1.4 Where the vocabulary lives in mini-swe-agent

Every term defined above corresponds to something *mini-swe-agent* actually contains, with one exception: the agent function is an abstract description of behavior and has no counterpart in the source. Table 2 gives the correspondence, and Figure 4 writes the same names onto the machine itself.

Two of those correspondences do the most work. The agent program is not one component but two: the controller, which proposes the next action, and the runtime’s control loop, which executes it, updates state, routes, and stops. Beneath both sits the architecture, which is not part of the program at all but the process the run lives in, the model client the controller calls, and the tool implementations that turn a requested action into a real one. Every version that follows, from the chain to the reflection step, changes what the controller is asked to decide, and none of them changes who executes. Working state is the second: it holds what the agent has seen so far, and how that record is declared and updated is taken up in Part B.

Covered in these notes only: §2.1, §2.2, and §2.3.1. The state representations of §2.4.7 are taken up in Part B with the state schema. Quotations, figures, and page numbers are from the Global Edition, in which the chapter occupies pages 54 to 79 and the assigned sections pages 61 to 74; the summary on page 78 is a fair self-test.¹

Looking ahead. The rest of these notes walk the question one decision at a time, on mini-swe-agent, each version a runnable program rather than a diagram. A *fixed workflow* (Section 2) makes every control decision at design time and lets the model fill in content. It is an agent without autonomy, rational only while the designer's prior knowledge holds, and Agent-less [19] is one published realization of it. A *reactive loop* (Section 3) computes the next action from the percepts so far, which is what a partially observable environment demands; ReAct [21] is its realization. A *planner* (Section 4) puts the intended sequence in state, where it can be inspected and revised. A *validator* (Section 4.5) hands acceptance to a judge that is not the proposer. A *reflection step* (Section 4.8) changes the next attempt's context without changing the model. Section 5 places these five, and Google's design patterns [5], on the taxonomy of agent programs [14, §2.4].

2 Fixed workflows

The upshot

1. **A fixed workflow is defined by who decides, and when.** We settle every control decision at design time: which operation runs next, how many model calls the run makes, and what ends it. The model is called at points we choose, and what it returns is content that travels through the graph without changing the graph.
2. **The fixed shape is what we buy.** One shape per run means the cost is known before the run starts, a failure belongs to the stage that produced it, and the run ends because the graph ends rather than because a counter ran out.
3. **The fixed shape is also what we pay.** Our environment is partially observable, so the evidence a decision needs may not exist until an action produces it. A graph with no edge back has nowhere to ask a second question, and the question it cannot ask is the one its own output depended on.
4. **The sections that follow give these decisions away one at a time.** Section 3 hands the choice of the next action to the model, Section 4 puts the intended sequence into state where it can be revised and moves acceptance off the component that proposed the work, and Section 4.8 lets one attempt change the context of the next. Every move costs something the fixed shape was giving us for free, so every move has to be earned.

2.1 Control decisions and content decisions

Definition 2.1 (Fixed workflow). A *fixed workflow* is an agent program in which we make every control decision at design time. The operations, the order they run in, the number of model calls, and the condition that ends the run are all written before the program sees a task. The model supplies content at points we choose. Such a program sits at one end of an axis the definition of rationality already gave us: “To the extent that an agent relies on the prior knowledge of its designer rather than on its own percepts and learning processes, we say that the agent lacks autonomy” [14, §2.2.3].

Two kinds of decision run through any agent program: i) control decisions and ii) content decisions.

A *control decision* selects what the program does next: which operation runs, whether anything runs after it, and when the program stops. A *content decision* fills in a value that an operation needs: which file looks suspicious, what a patch says, which lines are worth reading.

The distinction is not about difficulty. What I mean is, naming the file to patch is the harder problem (and the model decides that for us). Deciding that a file is named before lines are read is the one that fixes the shape of the run (and that's something that we do). Table 3 sorts the decisions of an agent program on that line.

Table 3 The decisions an agent program makes, and where a fixed workflow puts each one.

Decision	Kind	Who decides	When
Which operation runs next	Control	We do	Design time
Whether another operation runs after this one	Control	We do	Design time
How many model calls the run makes	Control	We do	Design time
What ends the run	Control	We do	Design time
Which artifacts a step reads and writes	Control	We do	Design time
What a model call returns	Content	The model	Run time
Which values travel to the next step	Content	The model, within our schema	Run time

Fixed does not mean straight. A fixed workflow may branch, filter, and repeat a step a set number of times; what makes it fixed is that we wrote the predicate on which it branches, so the set of paths through the program is closed and we can enumerate it. The distinction that matters is not one of graph topology but of authorship: **in a fixed workflow no model output is ever read as an instruction about what to do next. It is read as data.**

¹The United States printing of the fourth edition paginates the chapter as pages 36 to 63. Section and figure numbers are the same in both printings.

2.2 What we gain and what we lose

Fixed workflows can be a double-edged sword. They give us predictability and control, but at the cost of flexibility and adaptability.

2.2.1 What we gain

Three guarantees come with fixing the control decisions: i) the run has one shape, ii) a failure has an address, and iii) the run terminates by construction.

- **Shape.** The run has one shape, so its cost is countable before it starts. The model calls are the ones we wrote, and their number does not depend on what the model says. We can put a token budget on a task before we have read the task.
- **Failure mode.** A failure has an address. Every intermediate artifact is produced by a named step, so a wrong answer belongs to the step that produced it, and debugging a bad patch means reading four artifacts rather than reconstructing a forty-turn trajectory.
- **Termination.** The run terminates because the last operation returns. That is a fact about the program text and not a claim about the model's behavior (and it is the one guarantee the later versions never get back: they replace it with a model-call budget, which guarantees stopping but says nothing about finishing).

Our prior knowledge is doing all of this work, and doing it legitimately: “As a practical matter, one seldom requires complete autonomy from the start: when the agent has had little or no experience, it would have to act randomly unless the designer gave some assistance” [14, §2.2.3]. A fixed workflow is the limiting case of that assistance, where the designer supplies the whole program and leaves for the model only the words.

2.2.2 What we lose

The tradeoff is that a fixed workflow relies on assumptions we made when we designed it. If new evidence shows that those assumptions were incomplete or wrong, it cannot change its sequence of operations to investigate. This limits the autonomy a rational agent can have [14, §2.2.3]: “A rational agent should be autonomous—it should learn what it can to compensate for partial or incorrect prior knowledge”.

What I mean is, it is rational exactly while our prior knowledge holds, and when that knowledge turns out to be partial, the workflow does not notice. It has no representation of what it does not know.

The shape of that failure is specific. Our environment is partially observable (Section 1.2), so a rational agent here acts to see, and a fixed workflow does act to see *but* it does its looking on a schedule we wrote before we knew what the looking would return. If the value the patch depends on comes from a helper the alert never named, no node exists in which to ask for it. The chain does not stall and it does not report a gap. It writes the patch anyway, because that's how we designed the workflow.

Now, all of this is not to say that a fixed workflow is useless; my point simply is that it has obvious limitations that stem from its reliance on prior knowledge. In fact, Section 2.3 discusses a concrete approach that actually demonstrates a successful realization of a fixed workflow. After that, the next few sections will explore more autonomous agents and see how they address the limitations of fixed workflows.

2.3 Agentless, one realization

So far, our discussion has been about shape. Now let's see a system with that shape, and it is not a toy. Agentless resolves real GitHub issues by running three phases in a fixed order. It's a relatively older piece of work (October 2024) but it's worth looking at. Notably, it resolved more SWE-bench Lite issues than any open-source agent at that time, but at a fraction of their cost [19, §5.1].

The task is SWE-bench Lite: 300 problems, each one a real issue report and the Python repository it was filed against, where the job is a patch that resolves the issue [7]. That is the job `mini-swe-agent` does, on a different fixture.

Three phases, and the split from Table 3 is visible in every one of them. The model fills in content; we fixed the order, the inputs, and the counts.

- **Localization.** Three steps, each with a smaller input than the last. The repository structure (directories and file names, no source text) yields the top three suspicious files; a skeleton of those files (class and function headers, no bodies) yields the elements worth reading; the full source of only those elements yields the edit locations [19, §3.1]. The model ranks and selects at every step, and it never asks for something the step did not hand it (Figure 6).
- **Repair.** Each set of edit locations is widened by ten lines on either side, and the model writes a patch in a search-and-replace format borrowed from Aider [4]: the lines to find, and the lines to put in their place (Figure 7). Four location sets, ten patches each, so forty candidates per issue [19, §3.2, §4].
- **Patch validation.** The model writes a test that reproduces the issue, and then the runtime does the choosing. Candidates run against the repository's own passing tests, the survivors run against the reproduction test, and the winner is the patch that occurs most often once the survivors are normalized [19, §3.3].

In that third phase, acceptance is a test outcome and a vote, not a model's opinion of its own work, so the thing that proposes a patch is not the thing that accepts one. Section 4.5 is about that separation, and the numbers say it pays: majority voting alone resolved 77 issues, adding the repository's regression tests took it to 81, and adding the generated reproduction test took it to 96 [19, §5.2.3]. Agentless resolved 96 of 300 problems (32.00%) at an average of \$0.70 and 78,165 tokens per issue. SWE-agent, running the same model, resolved 55 (18.33%) at \$2.53 and 498,346 tokens [19, Table 1]. That pair comes closest to isolating control structure: hold the model fixed, change who decides what runs next, and the pipeline resolved more issues at about a quarter of the cost and a sixth of the tokens. The two systems differ in more than their control structure, so this is the strongest available comparison rather than a controlled experiment.

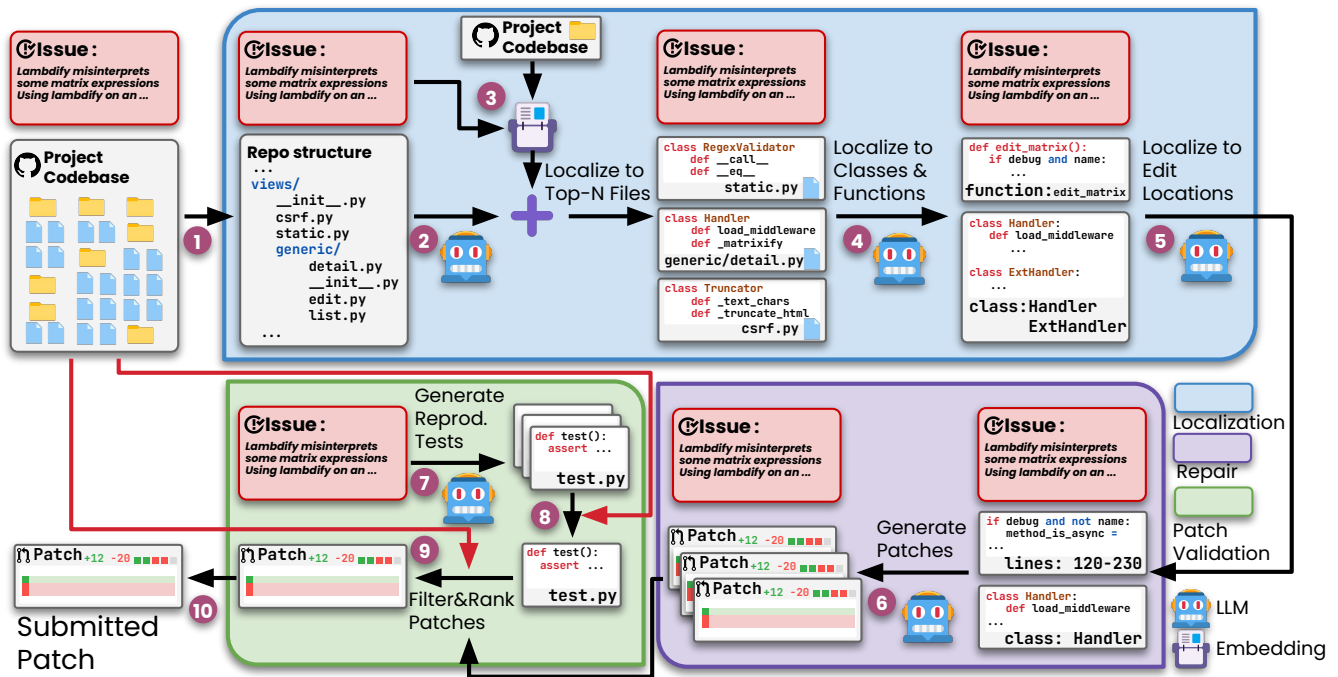


Figure 5 Agentless: localization (blue), repair (purple), and patch validation (green). The numbered steps are the whole program, and there is no arrow that a model output can redirect. Figure 1 of Xia et al. [19], reproduced from the authors' arXiv source (CC BY 4.0).

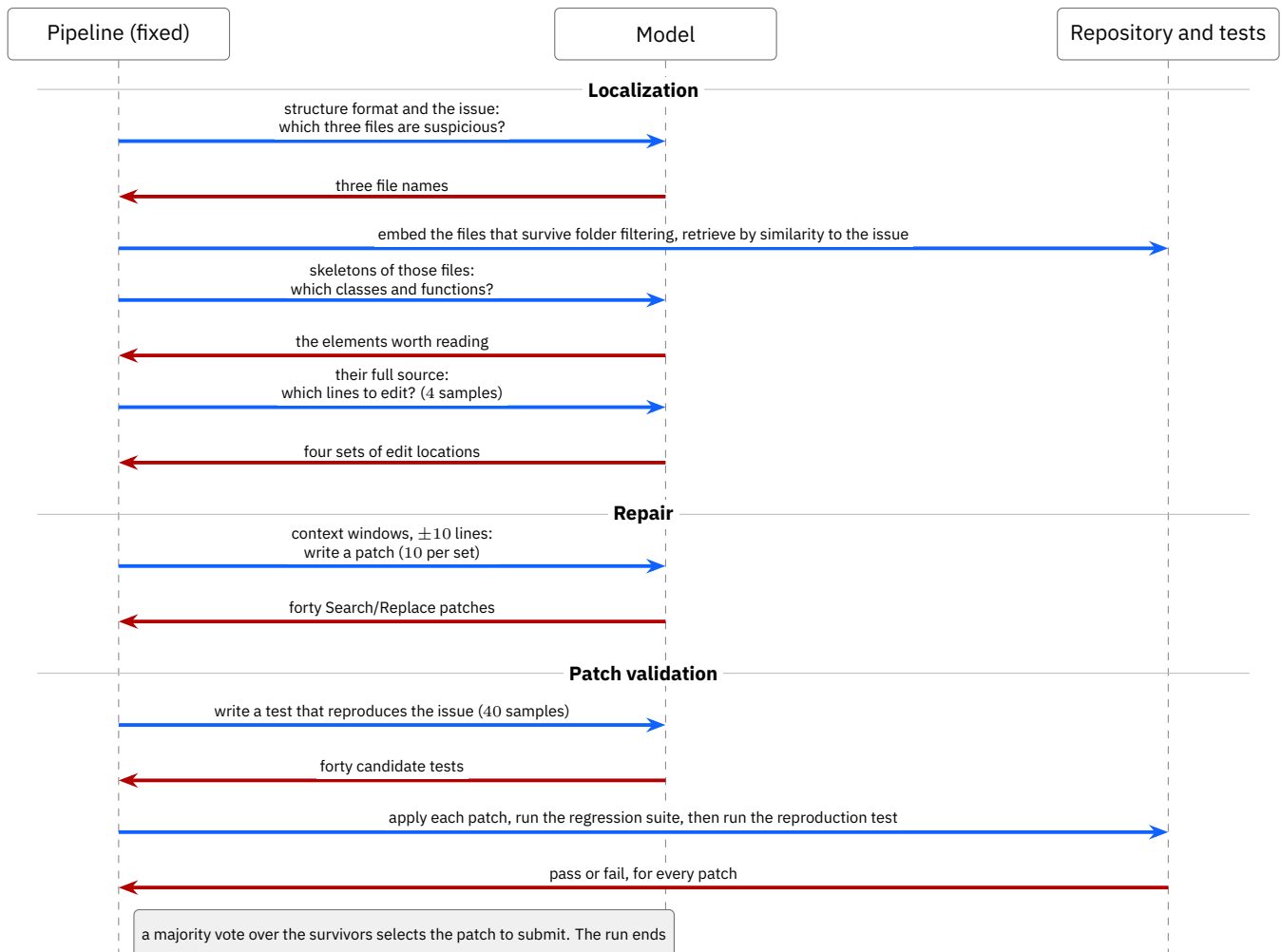


Figure 6 One Agentless run, with the paper's default counts [19, §3, §4]. Blue arrows are calls the pipeline makes; red arrows are what comes back. Read it against Figure 3, which uses the same colors for the same things: there every blue arrow leaves the agent, here not one of them leaves the model.

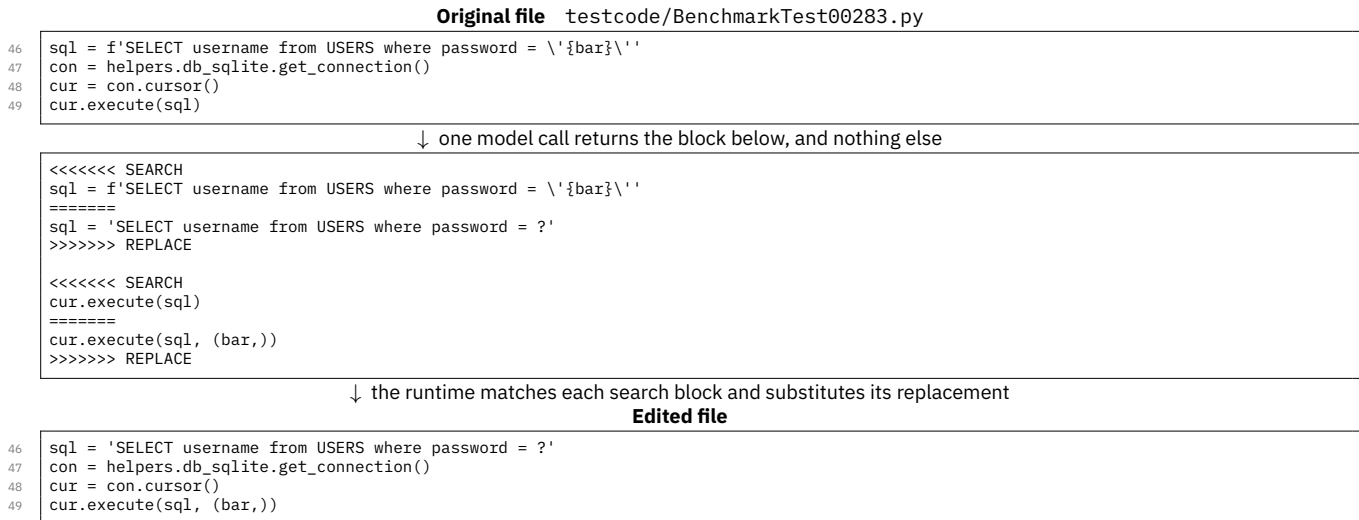


Figure 7 The Search/Replace edit format, on the worked alert of [Section 2.3](#). The model returns the middle block and never touches the file; the runtime matches each search block and substitutes its replacement. Small edits are cheaper than regenerating the region and leave less room for invention. Redrawn after Figure 3 of Xia et al. [19].

I'd take all of these results with a grain of salt. Two limitations qualify this comparison.

- First, Agentless did not lead the full leaderboard: several closed-source systems scored higher, including CodeStory Aide with 129 resolved issues (43.00%). Those systems released neither code nor execution trajectories, so we cannot examine how their control structures contributed to their results [19, Table 1].
- Second, SWE-bench Lite has problems as an evaluation set: 4.3% of its issue descriptions include the exact developer patch, 5.0% suggest a solution the developers did not use, and 10.0% lack enough information to solve the issue [19, §6.1]. The authors also report a filtered subset of 249 problems that excludes these cases [19, §6.2].

The answer leakage in that second limitation motivates a precaution in our own fixture: we exclude the answer key. OWASP BenchmarkPython stores expected results alongside its test cases [13]. If an agent can read those results, it can obtain the expected answers without analyzing the code, so its score no longer cleanly measures the ability we want to evaluate.

In a nutshell, on well-specified issues, a fixed workflow with an execution-based selector matched or beat the agents of its day. What it does not show is that fixed workflows win in general. Our position in the course is narrower in that the design-time side of [Table 3](#) is a serious engineering position that we will try to systematically remediate with structured autonomy.

The same shape over our own fixture would be shorter still, and it is worth seeing why. A scanner alert already names the file and the line, so the two phases Agentless spends recovering them collapse into a runtime lookup. What remains is a chain of four steps: read the flagged region, ask the model which lines to change, ask it for the patch, and submit. Call that shape *the chain*. Its cost is two model calls and one read whatever the alert, its trace has the same four entries every time, and it terminates because the last step returns.

What the chain cannot do is visible in that list before anything runs. `search` is one of `mini-swe-agent`'s four tools and no step in the chain can call it, so when the value a query interpolates reaches the flagged line through a helper in another file, the chain patches what it was given. It does not stall and it does not report a gap; it writes a plausible patch from the region it has, and a check that the patch applies, touches the flagged line, and clears a rescan will still pass. [Section 3](#) is where that limit is lifted.

HW1 uses both shapes, for the reasons this section gives. Its intake stage, which runs the scanners and normalizes their alerts into one contract, is a fixed chain: known cost, named artifacts, failures located by stage. Its investigation loop is not, because triage is where the second question gets asked.

2.4 Exercises

1. Agentless picks its final patch by a vote over test outcomes. Use [Table 3](#). Say whether that is a control decision or a content decision, who makes it, and when. What would have to change for the model to make it instead?
2. Count the model calls the chain makes per alert, before any run. Now name one cost of the run that the count does not fix in advance.
3. A submission check that confirms the patch applies, touches the flagged line, and clears a rescan will still accept a patch that deletes the query outright. Write a fourth check that rejects it, then say why the chain cannot run your check with the tools it has.
4. Add a fifth step to the chain that calls `search` once on an identifier taken from the flagged line. Which decision have you moved to run time, which have you kept at design time, and what can still go wrong?

2.5 Reading

Before class: Xia et al. [19], §1, §§3.1–3.2, and Figure 1. These notes also draw on §3.3 for patch validation, §4 for the defaults, §5.1 and Table 1 for the comparison, §5.2.3 and Table 4 for the selection ablation, and §§6.1–6.2 for the benchmark analysis.

Read the FSE version (Proc. ACM Softw. Eng. 2, FSE, article FSE037). The arXiv preprint carries the “Agentless” title prefix and numbers its sections differently, so the section references above will not find the right text in it.

Looking ahead. Section 3 moves the first row of Table 3 to run time: the model chooses which operation runs next, and the graph grows an edge back. That is what a partially observable environment asks for, since acting is how the agent sees, and it is what buys the second question the chain cannot ask. It also spends all three guarantees we counted in Section 2.2. The cost of a run stops being countable in advance, a failure stops having a single address, and termination stops being a fact about the program text, so the loop arrives carrying a model-call budget, a terminal status, and a trace.

3 Reactive loops

The upshot

1. **Action selection shifts from design time to run time.** In the fixed workflow, the next operation is specified in advance (Table 3). In a reactive loop, the model selects the next action conditional on the percept history. The remaining architectural differences follow from this transfer of control.
2. **A thought modifies context without affecting the external environment.** The action space is extended to include language. A thought updates the context used for the next decision but produces no external state change or observation feedback.
3. **Adaptive evidence collection relinquishes three guarantees.** Unlike a fixed chain, a reactive loop can request additional evidence in response to intermediate observations. This flexibility precludes an exact advance count of model calls, complicates attribution of failure to an individual step, and requires an explicit bound to guarantee termination.
4. **Model failures and runtime failures require distinct controls.** The model may repeat prior thoughts and actions without progress or proceed on uninformative observations. A separate runtime failure arises when model-generated content is accepted as an observation. The runtime must therefore retain exclusive write access to the observation channel.

3.1 The action that only changes the context

Definition 3.1 (Reactive loop). A *reactive loop* is an agent program in which the model selects the next action from the percepts so far, and the runtime executes that action and returns an observation on which the next selection is conditioned. The action space includes language: “we augment the agent’s action space to $\hat{A} = A \cup L$, where L is the space of language” [21, §2], and an action drawn from the language part is a thought, which “does not affect the external environment, thus leading to no observation feedback”.

While a fixed workflow has one shape, a reactive loop has a shape that depends on what comes back from the model.

The cycle has four steps:

- **Decide.** One model call reads the context so far. It returns a thought, a tool request, or a submission.
- **Dispatch.** The runtime validates the request against the tool’s signature and then executes it. An invalid request yields an observation rather than a crash.
- **Observe.** The runtime writes what the tool returned into state, where the next decision will read it. The model never writes this field.
- **Route.** A predicate we wrote reads the state and picks one of three successors: to Submit if the decision was a submission, to a terminal status if a runtime limit has been reached, and back to Decide otherwise. Because that predicate reads only the fields the runtime wrote, the model cannot talk the run past those limits.

Figure 8 is the graph the demo compiles, and it has three nodes rather than four steps. Dispatch and Observe both happen inside tools, and Route is the predicate carried on the conditional edges rather than a node of its own. A thought passes through the same cycle with the middle two steps empty: nothing is dispatched, nothing is observed, and the only thing that changes is the context the next decision reads. That is worth being exact about, because a thought still costs a model call and still appears in the trace (and in Section 4.8 it will be the only part of a failed attempt that survives).

Table 4 puts the decisions of Table 3 side by side in the two architectures. **The model gains exactly one row, and that row is the whole difference between the two sections.**

Table 4 The same decisions as Table 3, in a fixed workflow and in a reactive loop.

Decision	Fixed workflow	Reactive loop
Which operation runs next	We do, at design time	The model, at run time
Whether another operation runs after this one	We do, at design time	The model, until a limit fires
How many model calls the run makes	We do, at design time	Bounded by the budget, otherwise unknown
What ends the run	We do, at design time	A predicate we wrote over state
Which artifacts a step reads and writes	We do, at design time	We do, at design time
What a model call returns	The model	The model

The last two rows do not move, and that is the part worth holding on to. We still write the tool signatures, the state schema, the routing predicate and the runtime’s limits, so the model selects its next action from a set we closed in advance. A reactive loop is not an agent we have stopped programming; it is an agent whose order of operations we have stopped writing down.

3.2 What we gain and what we lose

The loop trades certainty for reach: we can no longer say what a run will do, and the agent can now go and get what it needs.

3.2.1 What we gain

Three things come back to us in exchange for the one decision we gave away.

- **Reach.** The agent can act to see. When the flagged line names a helper, the model can call `search`, read what comes back, and only then decide. The chain has no node in which to ask that question, and a loop needs no new node, because the edge back is the node.
- **Proportion.** The cost of a run now follows the alert: a two-line fix costs two or three turns, and a fix that crosses three files costs more. The chain pays the same price for both, because we fixed that price at design time.
- **Recovery.** An unhelpful observation is no longer fatal, because a search that returns nothing can be followed by a different search. The chain carries a bad read all the way to the patch.

3.2.2 What we lose

The three guarantees of [Section 2.2](#) go, and they go together.

- **Shape.** We cannot count the cost of a run before it starts, because the model-call budget gives us a worst case and nothing more. What we allot to an alert is now an allowance against that ceiling rather than a price for the work.
- **Address.** A failure is spread over turns, each of which can look reasonable on its own while the run still ends in the wrong place, so finding the turn that went wrong means reading the whole trajectory.
- **Termination.** The run ends because we made it end. The cap guarantees that the run stops but not that it finishes, and a run that exhausts its budget has no patch to show for itself.

Two failure modes arrive with the loop, and both belong to the model: it can repeat its own previous thought and action without escaping the repetition, and it can act on an observation that carries nothing, then treat the next one the same way.

Neither failure is wholly invisible to the program, and a runtime can guard against the degenerate case. Ours does: when the last three observations come back with the same result, the run stops with the status `no_progress` ([Section 3.4](#)). **What no rule of that kind can catch is a run in which something new comes back every turn and none of it helps.** Novelty is observable and progress is not, so the loop counts turns.

That gap is what [Section 4](#) exists to fill: a plan held in state gives the run something to compare a turn against, and a validator gives it a judge other than the model that proposed the work. [Section 3.3](#) reports how often these failures occur in practice.

3.3 How a reactive loop fails

The reactive loop takes its name from ReAct [\[21, §2\]](#), which prompts a frozen model with a small number of worked trajectories so that thoughts and actions are produced in a single stream. Here, the agent conditions subsequent reasoning and action selection on observations returned by external tools or the environment, rather than relying solely on model-generated claims. Such observations, while they provide evidence against which claims can be checked, do not guarantee correct interpretation or a correct answer.

Access to external evidence does not eliminate failure; fixed workflows and reactive loops exhibit different failure distributions. [Table 5](#) summarizes a manual annotation of 150 failed issues on SWE-bench Verified across three tools using the same backbone model, Claude 3.5 Sonnet [\[11, §3.2, §6.1\]](#). Holding the backbone model fixed removes model identity as a source of variation, but does not isolate the causal effect of architecture: the tools may also differ in prompts, interfaces, and implementation. The results therefore support a descriptive comparison of these systems rather than a causal attribution to architecture alone.

Table 5 Where failures appear, by architecture. Each share is of that tool's own failed issues, from 150 hand-annotated failures on SWE-bench Verified with one backbone model, Claude 3.5 Sonnet [\[11, §6.1\]](#).

Tool	Architecture	Stage where the failure appears	Share
Agentless-1.5	Fixed workflow	Localization	51.3%
OpenHands-CodeAct-2.1	Reactive loop	Iteration and validation	49.1%
Tools + Claude 3.5 Sonnet	Reactive loop	Iteration and validation	51.7%

A fixed workflow fails where it commits earliest: over half of its failed issues carry a localization error, and because no later node can revisit that choice, the remainder of the run elaborates a mistake it cannot see. The reactive loops fail instead inside the iteration, a stage the fixed workflow does not possess at all, which is to say that the machinery bought in [Section 3.2](#) is also where the new failures live.

Roughly 65% of the failures in that study originate in flawed reasoning rather than in missing information or environmental friction, and within a loop that flaw takes three named shapes [\[11, §6.1, §6.2\]](#).

- **Non-progressive iteration.** The tool repeats modification cycles, regenerating near-identical patches that differ only trivially while the defect persists. It is a loop that runs without going anywhere, and [Section 4](#) answers it by giving the run an explicit plan against which a turn can be measured.
- **Validation retreat.** The tool edits the failing test rather than the code, so that the test passes. The proposer is grading its own work, and [Section 4.5](#) answers it by handing acceptance to a judge that did not write the patch.
- **Context amnesia.** The agent loses its original objective over a long interaction. [Section 4.8](#) answers it by giving an attempt memory that outlives the run.

One of the three belongs to us rather than to the model. Unless the runtime owns the observation channel, the model will supply the observations itself: “soliloquizing, where the model self-generates hallucinated observations without interacting with the environment” [1]. It affected 48.4% of trajectories on one capture-the-flag benchmark with one model, while two other models produced none of it [1, Table 4]. **The runtime must write the observation field, and the routing predicate must read only the fields the runtime wrote.**

The other two are failures of judgement over a whole run, and they cost more than a wasted turn. Failed issues consume roughly 3.5 times as many interaction steps as successful ones, and where one tool covers 80% of its successes within 25 rounds, it requires 54 to cover 80% of its failures [11, §4.3, §6.2]. Every rate here comes from a single benchmark and a single backbone model, so what transfers is the shape and not the number.

A loop cannot tell which of those two runs it is in, because it never wrote down what it was trying to do. It has its own thoughts in the transcript and its observations in state, but nothing that says what the next three steps were supposed to achieve, so there is nothing for a turn to fall short of. Section 4 puts that intention into state, where the run can be measured against it and, when the evidence contradicts it, revised.

3.4 The demo: the loop

The loop is a runnable program, `demo/lec2_reactive_agent`, sitting beside these notes. One alert runs it:

```
python -m lec2_reactive_agent --alert-id bc284c6b
```

That is the worked alert of Section 2.3. The architecture is one file of about two hundred lines, `graph.py`, and Figure 8 is a picture of it: every box in that figure names the method that implements it and the lines it occupies.

The file defines one class, `AgentDAG`. Each node of the graph is a method on it, each routing predicate is a method on it, and compile wires them together. Nothing else in the program decides anything.

- **controller, lines 89 to 122.** The only node where a model decides. It does two things before it decides anything: it stops the run if the model-call budget of forty is spent, and it stops the run if stalled reports that the last three observations came back the same. Only then does it issue a call. Both limits are checked *before* the call rather than after it, because a limit checked afterwards is a post-mortem and not a control.
- **tools, lines 142 to 158.** Runtime work. It executes every call the model requested and returns an observation for each one, including the ones that failed. A failed call still gets an answer, because a model given no answer writes its next message malformed.
- **submit, lines 159 to 178.** Runtime work, and deliberately dull. It runs the three checks of Section 2.3 and routes on the outcome: accepted ends the run, rejected returns the reasons to the controller as feedback. No model judges acceptance here, which is precisely what makes a validator worth adding in Section 4.5.
- **after_controller, lines 179 to 188.** The routing predicate, and the one row Table 4 hands to the model. It branches on the run’s status and on whether the last message carried a tool call, and on nothing else. It never reads the model’s prose, so no amount of arguing inside a thought moves the run.
- **after_submit, lines 189 to 191.** Three lines: end, or go back to the controller with the rejection.
- **compile, lines 192 to 206.** The wiring. Two conditional edge sets and one unconditional edge, and it is the unconditional one that matters: the return from `tools` to `controller`. The comment beside it in the source calls it the one line that makes the version reactive. Delete that line and the program is a chain again.

What `graph.py` does not contain, it calls. The state record and its reducers are in `state.py`, the three tool signatures in `tools.py`, the validate-execute-observe sequence in `dispatch.py`, the three submission checks in `submit.py`, and the terminal statuses and the trace writer in `trace.py`. Section 4.8 and Part B take those up one at a time; for this section they are collaborators the DAG calls, and the DAG is the architecture.

One run. Figure 9 is a recorded trajectory, rendered from the run’s own trace by `trajectory.py`: four model calls, three tool calls, forty-seven seconds.

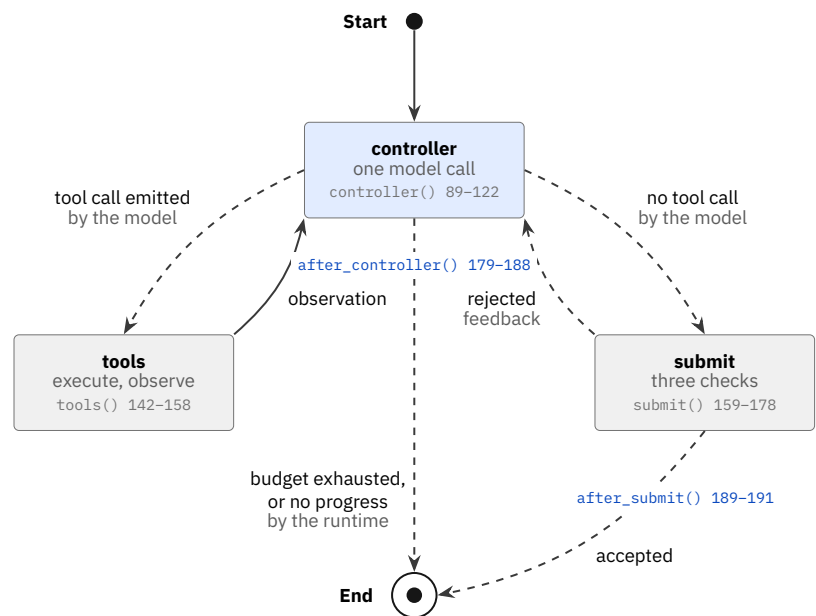


Figure 8 The DAG the demo compiles, with every component named by the code that implements it: method and line range in `demo/lec2_reactive_agent/graph.py`. Blue is the only node where a model call decides, grey is runtime work, and the two predicates in blue type are the conditional edges. The one solid edge, from `tools` back to `controller`, is what makes the loop a loop.

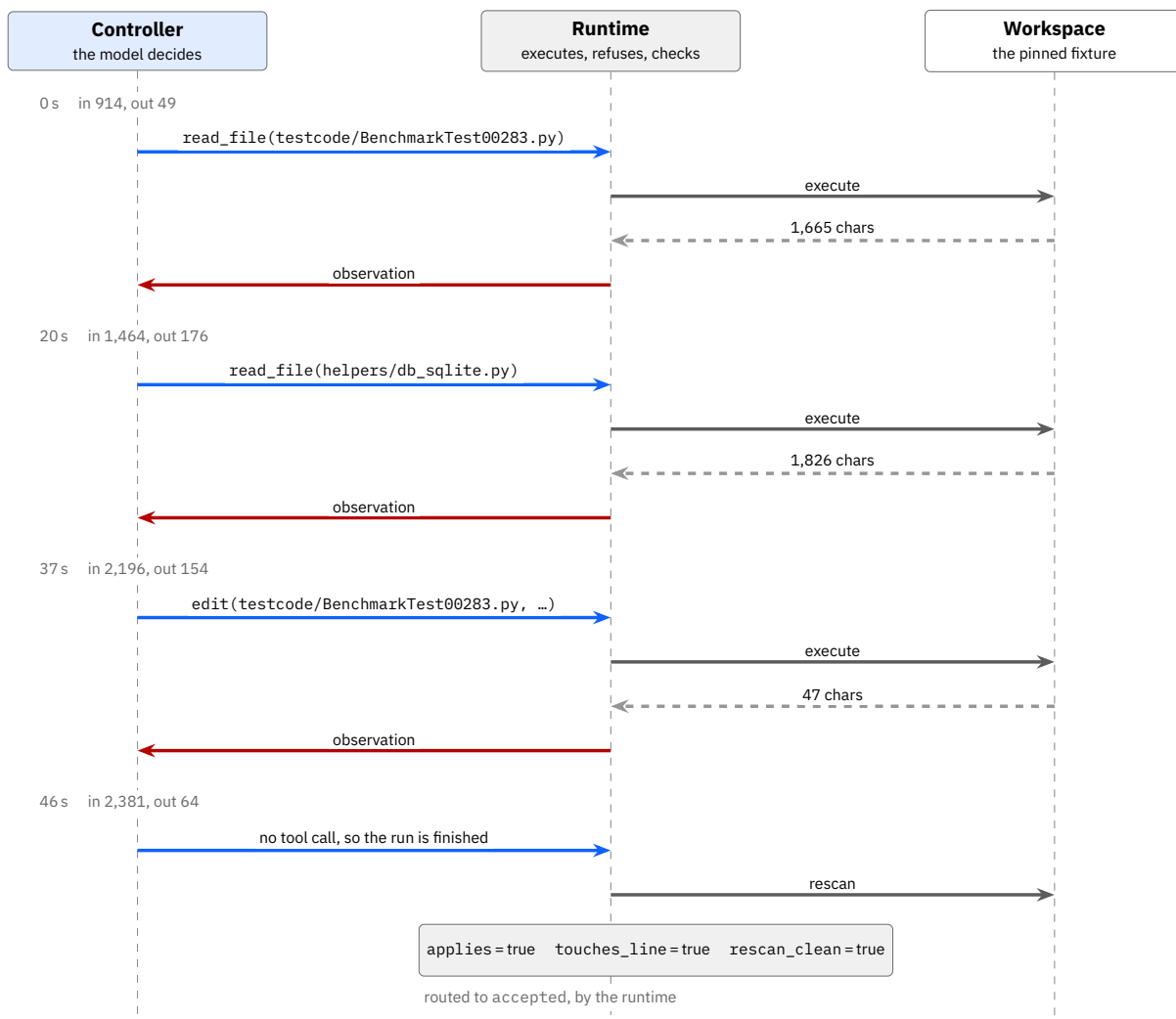


Figure 9 One recorded run on the worked alert, from `runs/v1-bc284c6b.jsonl`. Blue arrows leave the controller, red arrows return to it, and grey arrows are the runtime working against the fixture. Nothing reaches the controller that the runtime did not put there.

Three things in that trajectory are worth naming. At twenty seconds the model reads `helpers/db_sqlite.py`, a file the alert never mentioned, and that is the second question [Section 2](#) could not ask. The token counts stay in their own columns throughout, so the last call reports 2,381 input and 64 output rather than one number that conceals which of them grew. And the run ends because `submit` passed three mechanical checks, none of which asked whether the endpoint still returns what it returned before.

What the trajectory does not contain is the point. It holds four thoughts, one for each model call, and no plan. Nothing recorded at zero seconds says what the run intended to have done by forty-seven, so nothing later in the run can fall short of it.

4 Planning, validation and reflection

The upshot

1. **Intent held in state makes progress checkable. Reader judgement becomes a runtime predicate.** A plan records what the run is for, and a completion condition attached to a step converts progress from a judgement a reader makes into a predicate the runtime evaluates.
2. **Feedback changes a run at two levels.** An observation that agrees with the plan changes the next action alone; an observation that contradicts it changes the remaining plan. Rewriting the remaining plan is the event a planner exists for, and counting ordinary progress as one makes a planner expensive without making it useful.
3. **Independent acceptance requires both a separate check and a trustworthy signal.** Asking a model to “check its work” supplies neither independent control over acceptance nor external evidence of correctness. Without an external signal, self-assessment can lower accuracy while adding calls. The workflow therefore gives mechanical checks control over acceptance and confines the model-based validator to rejection.
4. **A refused attempt leaves behind guidance, not its context.** Reflection turns a refusal into text that outlives the attempt reset, and the reset is what separates a second attempt from a longer first one.

4.1 The plan as a field of state

Planning sounds like something that belongs at design time, and in the chain it did: the order of operations was the plan that we wrote. What changes here is that the run writes one for itself while it is running and then has to operate with what it wrote.

Definition 4.1 (Open-loop planning). A planning agent is *open-loop* when the whole plan over the horizon is fixed by the initial planning policy and never revised.

Definition 4.2 (Implicit closed-loop planning). A planning agent is *implicitly closed-loop* when the plan is held at its initial value and feedback modifies a single action: such a system maintains $P_t = P_0$ and “only modify[s] a single action based on the feedback” [17, §2].

Definition 4.3 (Explicit closed-loop planning). A planning agent is *explicitly closed-loop* when feedback revises the plan itself, following “the policy $\rho(P_t|g, c_t, P_{t-1})$, where $P_t \in \Delta(A^{T-t})$ is the refined plan at time step t containing the modified future actions $a'_{\geq t}$ to execute and P_{t-1} is the old plan modified in the previous time step” [17, §2].

Our reactive loop sits off this axis entirely, it has no plan to hold fixed or to revise. AdaPlanner files ReAct and Reflexion among the methods whose feedback changes the action rather than the plan, with no decomposition of the task into subgoals [17, Table 1]. And, to be honest, that is a fair description of what we built. The context grows with each observation, and the next action is chosen from it.

Now, the mechanism that makes an explicit plan worth holding is not the list of subgoals. It is what AdaPlanner attaches to each one, where “each subgoal ends with an assertion statement to test its fulfillment, which allows our method to interact actively with the environment and later resume its execution with a refined plan” [17, §3.1]. **And, an assertion is a completion condition written at plan time and evaluated by the runtime, which is exactly the object the loop was missing.**

An executable assertion separates two questions: who decides what counts as completion, and who checks whether that condition holds? Here, the planner writes the condition and the runtime evaluates it. Execution establishes whether the condition holds, not whether it captures the task’s actual requirements. An assertion is therefore still a signal, subject to the fourth claim above: every subgoal can pass its check while the alert that started the run remains unresolved. **The runtime checks the planner’s definition of completion; it does not make that definition correct.** The three checks in `submit`, by contrast, are conditions we wrote and the runtime evaluates. That is why [Section 4.7](#) rests on those checks rather than on the plan’s own assertions.

An explicit plan records subgoals, proposed actions, and completion conditions; it need not be executable code. For example, a plan can record a structured condition such as `status = resolved`, name a predefined check such as “all required tests pass,” or state a prose criterion such as “the response addresses every item in the request.” The runtime can compare an observed field with the structured condition or invoke the named check; the prose criterion instead needs to be translated into a check or judged by a model or a human. These are all ways to record what counts as completion, but only those tied to an executable check provide a runtime assertion without further interpretation. None guarantees that the chosen condition captures the task’s

actual requirements.

AdaPlanner realizes this general structure as Python code, with comments describing subgoals and code specifying actions and assertions [17, §3.1]. This representation makes both the proposed actions and their completion conditions inspectable, and lets the runtime evaluate the assertions rather than asking a model to interpret prose conditions.

Food for thought for HW1

Is the agent you are building open-loop, closed-loop, or somewhere between? Decide it by naming what your run does with an observation that contradicts what it expected: nothing, one action, or the rest of the work. Then write down what your plan has to record for that answer to be true, and which of the three forms above each completion condition takes. Last, write the condition that fires a revision before you write the node that performs one. A planner that revises whenever anything surprising arrives is an expensive way to track the observations.

A reactive loop asks the model to choose the next operation for the task as a whole. Adding a planner narrows that choice: the model chooses an operation for the current subgoal. The plan also records three things that the reactive loop does not explicitly track: the subgoals to achieve, the conditions for completing each step, and which parts of the plan to revise when an observation contradicts it. Adding a validator lets a model reject a candidate, while the runtime still controls whether it can be released. Planning, revision, and validation require additional model calls. Table 6 summarizes these differences.

Table 6 How a planner and a validator change the reactive loop described in Table 4. The first row shows how the choice of the next operation changes; the remaining rows compare goal tracking, completion checks, revision, acceptance, and cost.

Decision	Reactive loop	With a planner and a validator
Who chooses the next operation?	The model, for the whole task	The model, for the current subgoal
Where are subgoals recorded?	No explicit plan	In the plan stored in state
What marks a step as complete?	No explicit completion condition	A condition recorded by the planner
What changes when an observation contradicts expectations?	The context	The context and the remaining plan
Who decides whether to accept a candidate?	The runtime applies fixed checks	The runtime; a model can also reject
What limits model calls?	The call budget	The budget must also cover planning and validation

Acceptance is the row where a validator gains least. A validator that can refuse a candidate is a different thing from a validator that can release one, and Section 4.7 defends keeping that difference.

4.2 Three shapes of planner

A planner is defined less by how it writes a plan than by what it does when the world disagrees with one. Three arrangements answer that differently, and each answers the same four questions: what the planner sends, what comes back, what changes when it does, and what the arrangement costs.

- **Open-loop.** The planner sends the whole action sequence at once and nothing comes back. Nothing changes, because there is no return path along which a change could arrive. One planning call buys the entire run, which is the cheapest of the three and the only one that cannot notice that its plan has stopped being right.
- **Implicit closed-loop.** The planner sends one action at a time and every observation comes back to it. What changes is the next action; the remaining actions stand as written, since the plan is held at the value it had when the run started. The cost is a model call per action, and the exposure is that an error in the original plan is carried to the end by a run that keeps adapting around it.
- **Explicit closed-loop.** The planner sends one action at a time, every observation comes back, and the observation is then sorted. One that matches what the plan expected feeds the next action and leaves the plan alone. One that contradicts it returns to the planner, which rewrites the remainder of the plan and names the step to resume from. The cost is a model call per action plus one per rewrite, and the rewrites are bounded by checking at one point per subgoal rather than at every turn, which holds a run to “the success status only at N crucial points” [17, §3].

A revision replaces the remainder of the plan rather than adding to it, and that is what separates revising a plan from accumulating context. Figure 10 draws the difference literally: in the third arrangement the actions from the current step to the end are struck out and a new tail is written in their place.

Our reactive loop accumulates. Every observation makes the context longer and nothing in it is ever withdrawn, so a conclusion the run reached at its second turn is still sitting in the transcript at its twelfth, arguing its case. On this axis the loop sits closest to the implicit arrangement with the plan removed: feedback changes the next action, and there is no remainder to revise.

The sorting step is where the assertion of Section 4.1 does a second job. An observation can be filed as agreeing or contradicting only where an expectation was recorded before it arrived, and AdaPlanner “can anticipate environmental observations and proactively refine the plan only when there is a discrepancy between expected and actual outcomes” [17, §3]. A completion condition closes a subgoal, and it also decides which return path an observation takes.

The skill memory in Figure 10 is the one component we do not carry over. It stores plans that succeeded and offers them back when a later task resembles an earlier one [17, §3.3]: memory that crosses from task to task. What Section 4.8 needs crosses

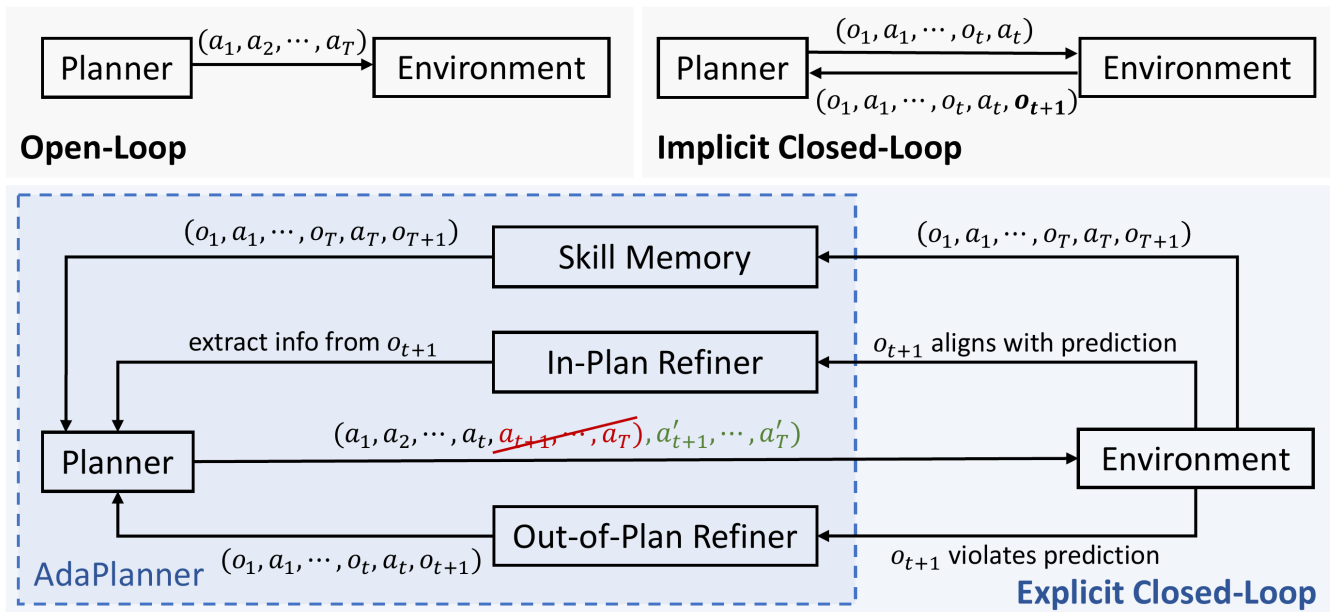


Figure 10 Open-loop, implicit closed-loop, and explicit closed-loop planning. The left arrangement sends actions and receives nothing. The middle one returns each new observation o_{t+1} to the planner but holds the plan at P_0 . The right one sorts o_{t+1} by whether it aligns with the prediction or violates it, routes it to the in-plan or the out-of-plan refiner accordingly, and in the second case strikes out the remaining actions $a_{t+1}, \dots, a_T$ and writes $a'_{t+1}, \dots, a'_T$ in their place. Feedback reaches the planner in both closed-loop arrangements; what the explicit one does with it is revise the plan rather than one action, which is the difference [definition 4.3](#) turns on. Figure 1 of Sun et al. [17], reproduced for teaching with credit (arXiv non-exclusive licence).

from attempt to attempt inside one task. The boundary is different, so we build an attempt memory rather than borrow the skill memory.

4.3 Revising plans

Not every unexpected observation calls for a new plan. Rewriting the plan after every surprise costs a model call each time and turns it into a record of events rather than a guide to action. Before implementing a response, we therefore need to distinguish four cases. Only one revises the plan within the current run.

- **In-plan.** The observation agrees with what the plan anticipated, and what the run takes from it is information the next action needs. AdaPlanner does this with a dedicated action that queries the model to parse the observation, so the extracted value flows into later actions [17, §3.2]. The plan itself is untouched. This is the loop of [Section 3](#) with somewhere to put what it learned.
- **Out-of-plan.** A subgoal's assertion fails, and the planner regenerates the remaining plan from the current state. AdaPlanner assembles an error message reporting where the agent is, what it holds, and its last three interactions, then refines the plan and sets the step to resume from, so the episode continues from an intermediate point rather than restarting [17, §3.2]. This is the one that changes what the run intends to do, and the mechanism a planner is built around.
- **Neither.** On a fixture as small as ours this is the common case. When the model follows a value into a helper file and finds what it needed, no assertion has failed and the plan has not been revised: a subgoal has been completed, in the ordinary way, by the step the plan said would complete it. **Ordinary progress is not replanning, and counting it as such makes a planner look busy while it is doing nothing.**
- **Not in this run.** A whole attempt is refused, and a fresh plan is written with what the failed attempt taught. That event ends the run rather than redirecting it, so it belongs to [Section 4.8](#), along with the memory that has to survive the reset.

4.4 Does planning make a difference?

Planning is something that is easy to buy into but hard to measure, because a system that plans usually differs from a system that does not in more ways than the planning.

The question to answer is: does planning improve performance when everything else is held fixed? Recent work by Wang et al. [18] tests this question directly. The researchers compare four ways of choosing the next action: one step at a time, beam search, lookahead, and lookahead with value propagation, keeping the rest of the agent setup the same.

Planning helps, and the gain is not reserved for large models. The smallest model tested, planning ahead, outscores a frontier model that chooses one step at a time.

- On one benchmark, the smallest model tested, with 8 billion parameters, scores 73.8% when it chooses one step at a time. Its score rises to 85.6% when it looks several steps ahead and uses estimates of future outcomes to choose its next action, a method called lookahead with value propagation.
- That is higher than the 78.3% scored by a frontier model (at that time) choosing one step at a time [18, Table 1].

The four rows of [Table 7](#) differ only in what an action is scored against. Choosing one step at a time scores the action itself, and

beam search keeps more candidates while scoring them the same way, so it is wider rather than different in kind. Lookahead scores an action by simulating the trajectories it leads to and reading off those outcomes instead.

Value propagation is what the fourth row adds. Each simulated outcome is pushed back onto every action along the way, so an action's value aggregates the futures that run through it, and evidence from late in a simulation can revise a choice made early. Lookahead without that feedback is not enough on its own: "Lookahead alone is insufficient unless information from simulated futures can revise early decisions" [18, §4.2].

Table 7 is a more illustrative view because it measures the failure rather than the success.

Table 7 How four ways of choosing the next action fail, aggregated over three benchmarks. The last row is the paper's Flare. Lower is better in the first column, higher in the other two [18, Table 3].

How the next action is chosen	Dead end on move one	Step of first error	Recovers after it
One step at a time	55.6%	1.6	5.4%
Beam search	71.9%	2.0	11.4%
Lookahead	23.6%	2.8	22.4%
Lookahead with value propagation	17.8%	3.2	29.7%

Read the top row against Section 3.3. A run that chooses one step at a time walks into a branch it cannot come back from on its very first move more than half the time, makes its first error before the second step, and escapes that error once in twenty. That is the loop which repeats itself without going anywhere, and these three numbers say what the trajectory could not: the error arrives almost immediately, and the run then cannot get out.

What makes a plan good is what follows from it, and neither more search nor a more human-sounding plan is the same thing. The remaining rows, and the two studies after them, are three ways of finding that out.

Now read the beam-search row, which is the surprise. Beam search searches more than choosing one step at a time, and it walks into the first-move trap more often. More search over the same local score does not help, because the score is the problem. What moves the numbers is evaluating an action by what follows it.

Two results say the representation is part of the mechanism. Stripping only the code interface from a planner and writing the same plans in language costs it 81% against 46% [17, Figure 4c]. Independently, web agents given plans in a structured formal representation beat the same agents given free-form plans at i) subgoal completion, 70.3% against 63.2%, ii) plan completion, 37.5% against 34.6%, and iii) task success, 36.4% against 32.1% [2, Figure 3]. One round of revision after exploring the environment then carries structured-plan subgoal completion from 70.3% to 93.3%.

One finding in that study separates a good plan from a plan that looks right. Revision moves the agent's plan *away* from the human-written plan while improving what the agent achieves, with exact matches against the human plan falling from 67.7% to 59.0% [2, Table 2]. Resemblance to the plan a person would have written moves in the opposite direction to the thing we are trying to improve.

Four things bound all of this.

- **Task.** Neither study is program repair. One is knowledge-graph question answering, the other is live web navigation.
- **Environment.** The failure numbers come from "oracle-structured subgraphs that guarantee the existence of valid solution paths" [18, §5.1], so the environment is deterministic, fully observable, and known to be solvable. Ours is none of the three, and for some alerts no patch exists at all.
- **Mechanism.** Simulating candidate futures and scoring them is not what our planner does. What it shares with writing subgoals and completion conditions is only that early decisions are made to answer to later evidence, so it validates the family and not the member.
- **Judge.** The web-agent task success rate tops out at 45.1%, its errors sit mostly below the plan rather than in it, and it is scored by a model judge of the kind Section 4.7 spends its length warning about [2, §4.2, §5.2].

What survives those bounds is the shape rather than any rate. A run whose early actions answer only to local scores commits early and cannot recover, and giving those actions something later to answer to is what changes the outcome.

4.5 The proposer and the judge

Acceptance has been the runtime's job since the chain, and nothing so far has asked whether the runtime is any good at it. The three checks in `submit` are cheap and they are dull, and both of those turn out to be load-bearing properties rather than shortcomings.

Definition 4.4 (Actor and evaluator). An agent separates *proposal* from *assessment* when the component that produces a candidate is not the component that scores it. Reflexion draws the split as two of three models: "an Actor, denoted as M_a , which generates text and actions; an Evaluator model, represented by M_e , that scores the outputs produced by M_a " [15, §3]. The evaluator "takes as input a generated trajectory and computes a reward score that reflects its performance within the given task context", and it may be an exact-match grader, a heuristic written for the task, or another instance of a language model [15, §3].

Self-Refine draws the same picture with one model in all three roles. It generates an output, prompts the same model for feedback on that output, and prompts it again to revise, with three prompts and no training [12, §2]. What matters for us is

where its loop exits, because the stopping condition “either stops at a specified timestep t , or extracts a stopping indicator (e.g. a scalar stop score) from the feedback” [12, §2], and the feedback is written by the model whose work is being assessed.

Our loop exits on a value the runtime wrote. That difference does not show up in a diagram of boxes and arrows, and it is not something a prompt can establish. **The accepting decision belongs to whichever component writes the value the exit edge reads, which is a fact about the graph we compiled and not about anything we asked the model to do.** Instruct a model to be strict with itself and it remains the author of the number the edge tests.

So the question in front of us is not whether to separate the proposer from the judge, because `submit` has been separate from the controller since the chain. The question is what a second model adds to a separation we already have, and what it costs to let that model hold the decision.

4.6 What a model judge can and cannot see

Each check answers its question with a program rather than an opinion: i) the patch applies, so the edit is well formed; ii) it touches the flagged line, so the run addressed its alert; iii) the rescan is clean, so the scanner is quiet. Each is fast, each is reproducible, and none of them needs a model.

Here, a verifier is *sound* when it never accepts an incorrect solution, and *complete* when it never refuses a correct one. Soundness on its own buys nothing, because a verifier that refuses everything never accepts an incorrect solution. **Completeness is what carries the property the argument below rests on: a verifier that produces no false refusals leaves a system no worse off than not checking at all, because anything generated correctly is still accepted.** The external verifiers behind the gains in Section 4.7 are both at once, being decision procedures for their domains.

Our three checks are neither. A scanner that reports nothing has not established that nothing is there, and an edit that renames the flagged expression or moves it behind a wrapper can clear an alert while leaving the weakness in place, which is a false acceptance. A correct patch that fixes the weakness somewhere other than the flagged line fails the second check, which is a false refusal. What the checks are is cheap and reproducible, and the error to avoid is reading them as though they were sound. That gap is the validator’s job, and it is a narrow one. Given the alert, the candidate patch, the results of the three checks, and the trajectory that produced them, a validator can ask three questions the checks cannot.

- **Weakness or alert.** Does the patch remove the unsafe behavior, or does it remove the scanner’s ability to see it? This is the validation-retreat failure of Section 3.3 under a different name, and on this fixture it is the failure most worth catching.
- **Behavior.** Does the edit change what the endpoint returns? The performance measure cares and none of the three checks looks.
- **Provenance.** Did the run actually establish what its patch assumes, or did it assert it? The trajectory is in front of the validator and was never in front of the checks.

4.7 Why the judge does not decide acceptance

Having found something for a second model to do, we now have to decide how much authority it gets, and the evidence on that question is unusually one-sided.

Start with the setting the results are about. *Intrinsic* self-correction is the case in which a model revises “based solely on its inherent capabilities, without the crutch of external feedback” [6, §1], which is precisely the case where a validator holds the accepting decision and nothing else does. Measured that way on reasoning benchmarks, accuracy falls while cost rises: one model answers a grade-school mathematics benchmark at 95.5% in a single call, 91.5% after one round of self-correction across three calls, and 89.0% after two rounds across five [6, Table 3]. The same pattern appears on a commonsense benchmark, where a weaker model falls from 75.8% to 38.1% after one round [6, Table 3].

Supply the ground-truth label instead, so that correct answers are never revised, and the same procedure improves that mathematics result from 95.5% to 97.5% [6, Table 2]. The gain is real and it is not available: “If we are already in possession of the ground truth, there seems to be little reason to deploy LLMs for problem-solving” [6, §3]. Two systems whose reported improvements rest on that setup are named, and one of them is Reflexion [6, Table 1], which matters for how we read Section 4.8.

A second study separates the verifier from the generator and varies only the verifier. Across 100 instances per domain with one model, self-critique moves a graph-coloring result from 16% to 2% and a Game of 24 result from 5% to 3%. Replacing the model verifier with a sound one moves the same two results to 38% and 36% [16, Table 1]. On a blocks-planning domain self-critique does help, from 40% to 55%, and a sound verifier still beats it at 60% to 87% depending on how much detail the critique carries [16, Table 1].

The verifier’s error profile explains the collapse. Used as a verifier, the model reaches 72.4% accuracy on graph coloring, refusing 95.8% of the correct answers it saw and accepting 6.5% of the incorrect ones, and 79.6% on a disguised blocks domain, where those two rates are 97.09% and 0.5% [16, Table 2]. A judge that refuses nearly every correct answer does not merely fail to help, because “the system rejects most answers and then times out on a set of later, worse generations” [16, §5]. The acceptance error runs the other way. On the two domains where the judge does least badly it accepts 10.4% of incorrect Game of 24 answers and 18.55% of incorrect blocks plans, against 6.5% and 0.5% on the two where it refuses almost everything [16, Table 2].

One finding sets our design rule. Once the verifier is sound, binary feedback, feedback naming the first error, and feedback naming every error land within four points of one another on three of the four domains. Those readings are 38%, 37% and 34% on graph coloring, 36% against 38% on Game of 24, and 10%, 8% and 6% on the disguised blocks domain. On blocks planning they do not, running 60%, 87% and 83%, so there naming the first error is worth twenty-seven points over a bare refusal [16,

Table 1]. **What a loop needs first from its judge is a trustworthy accept-or-reject bit; what an attached explanation adds on top of that ran from nothing to twenty-seven points across the four domains.** Dropping the critique altogether and sampling repeatedly does at least as well on two of them, reaching 44% on graph coloring and 42% on Game of 24 at twenty-five samples [16, Table 1].

These are four reasoning and planning domains with one model family, not program repair, and repair has something those domains lack: a rescan and a test suite are real programs that run. What transfers is the error profile of the source rather than a rate we can borrow. Where the accepting signal comes from a model, the outcome follows that model's error rates rather than the idea. The loop collapses on the two domains whose false-refusal rates run above 95%, and on blocks planning, where that rate is 15.48%, it gains fifteen points where a decision procedure gains twenty to forty-seven [16, Tables 1 and 2]. Where the signal comes from a verifier that is correct in both directions, the loop improves on all four. Our three checks are programs and they are correct in neither direction, so what they inherit is reproducibility rather than the guarantee.

That is why the mechanical checks keep the accepting decision. **The validator may refuse a candidate that passed all three checks, and it may not release one that failed them.** Confined that way, the only error it can make is refusing work that was already finished, and a refusal costs a turn against a budget we set. Were it allowed to release, its false acceptances would end the run, and nothing downstream would look again. Given false-negative rates of the size measured above, a judge confined to refusal can still stall a run by refusing everything, which is what the attempt cap of Section 4.8 is for.

Section 2 and Section 3 both promised that acceptance would move to something that did not write the patch, and it has. `submit` is that something, and the validator is a second one standing beside it. Neither section promised that the judge would be a model, and the evidence above is why we do not make it one.

4.8 What survives a refusal

A refusal ends an attempt, and it need not end everything the attempt learned. What crosses that boundary is a design decision, and the two obvious answers are both wrong.

The loop itself is short. The validator refuses a candidate, the attempt limit permits another try, and the runtime starts the next attempt from a cleared record. Something has to change between the two, or the second attempt is the first one repeated at twice the price. What changes is the context the next attempt begins with, and we get to choose what goes into it.

- **The whole trajectory.** Too large to carry, and it is the accumulation we were trying to get out of. An attempt that inherits every observation of the attempt before it is a longer first attempt wearing a new label.
- **The patch.** The artifact that just failed. Carrying it invites the next attempt to defend it, which is the one thing we do not want a fresh attempt doing.
- **The thought.** The action that changes the context and touches nothing else (Section 3.1). It costs a model call, it has no effect in the environment, and it is the only one of the three that is smaller than what produced it.

Definition 4.5 (Reflection step). A reflection step converts the outcome of a failed attempt into text that conditions later attempts. Reflexion adds it as the third of its three models, “a Self-Reflection model, denoted as M_{sr} , which generates verbal reinforcement cues to assist the Actor in self-improvement” [15, §3]. Given “a sparse reward signal, such as a binary success status (success/fail), the current trajectory, and its persistent memory”, it produces feedback that “is then stored in the agent’s memory” [15, §3].

Two lifetimes come with that arrangement, and they are the same two our state record keeps: “the trajectory history serves as the short-term memory while outputs from the Self-Reflection model are stored in long-term memory” [15, §3]. The trajectory dies with the attempt; the reflection does not.

Our reflection node runs on rejection and writes one short entry from four inputs: the alert, the candidate patch, the validator's reasons, and the trajectory that produced them. The entry appends to reflections, which survives the reset, while the observations, the plan, the candidate and the validation are cleared for the new attempt. The counters survive too, so the budget of forty model calls still binds across both attempts rather than restarting with each one. **The reset is what makes the guidance worth anything, because guidance delivered alongside the context that produced the failure is just more context.** Which fields are cleared, and why clearing an appending field takes an explicit assignment, is the state machinery of Part B.

What this does not buy is a second opinion from nowhere. The signal our reflection step receives is the validator's rejection reasons, which are the three mechanical checks plus a model's objections, and Section 4.6 said what those checks are worth: they are neither sound nor complete. The measurements in Section 4.7 were about revision with no external signal at all, and this is not that case, but it is not the ground-truth case either. What we expect is improvement where the failure was legible in the reasons the run was given, and no improvement at all on an alert for which no patch exists. The attempt cap is there because neither outcome announces itself.

4.9 What we gain and what we lose

The planner, the validator and the reflection step are additions of one kind: none of them takes a decision away from us. We wrote the plan schema, the assertion format, the validator's authority, the routing predicates and the reset contract, and every one of those is still design time. What we hand over is a second and a third kind of content decision, and each arrives with its own kind of error.

4.9.1 What we gain

- **Measure.** Progress becomes a predicate. A turn can fall short of a condition the run itself wrote down, which is the thing [Section 3.2](#) said no rule over observations could supply.
- **Intent.** The plan is a readable field of state, so a person watching a run can see what it believes it is doing before it finishes doing it. The trajectory of [Section 3.4](#) held four thoughts and nothing a reader could hold the run to.
- **Address.** A failure recovers part of the address that [Section 2.2](#) gave us and [Section 3.2](#) spent: the subgoal whose condition failed names where the run stopped making progress, even though it does not name why.
- **Refusal.** A patch that passes all three mechanical checks can still be turned back, which closes the one gap those checks were always going to leave open.
- **Memory.** A refused attempt leaves guidance behind, so a second attempt starts from what the first one was told rather than from the empty context the first one started in.

4.9.2 What we lose

- **Calls.** Planning costs a call and every revision costs another. Against a fixed budget these compete with the executor's calls, so a run that replans twice has fewer turns left to act.
- **Commitment.** A plan the model wrote badly is now something the run is committed to, and a bad plan is more durable than a bad turn. The loop could abandon a bad idea by having a better one; a planner has to detect that the plan is wrong before it can replace it.
- **Judgement.** A second model is a second source of error, and its refusals are expensive in a way its silence is not. A validator that refuses good work spends the budget on work that was already done.
- **Attempts.** The attempt loop multiplies everything beneath it, because a second attempt pays for a second plan, a second set of turns and a second validation against the same budget. The guidance it starts from was written by the model whose attempt failed.

Of what [Section 2.2](#) called shape, address and termination, these three additions return neither shape nor termination, and they return the address only as far as the failed subgoal. We still cannot cost a run before it starts, the attempt cap replaces termination with a counter we set, and the accepting decision stays with the three mechanical checks, because neither the validator nor the reflection step is safe to hold it.

4.10 The demo: the planner and the validator

Both additions are one program, `demo/lec2_planning_and_validation_agent`, sitting beside these notes. One alert runs it:

```
python -m lec2_planning_and_validation_agent --alert-id bc284c6b
```

Its own README says why the two arrive together: the package holds two segments in one graph, both are always compiled, and so one run shows both. [Figure 11](#) is that graph, with every node named by the code that implements it.

Against the loop of [Section 3.4](#) exactly two edges changed, and each carries one of this section's arguments.

- **The edge from tools acquired a second destination.** In the loop it ran unconditionally back to the controller, and its own comment there called it the line that makes the agent reactive. Here `after_tools` chooses, and the second choice is the planner: when the recent observations have said nothing new, the runtime replaces the plan instead of ending the run. The loop ended such a run with `no_progress`.
- **submit can no longer accept.** It runs the same three mechanical checks and then routes on the outcome, and only `validate` reaches END with the run accepted. That the proposer is not the accepter is therefore a property of the compiled graph, and no prompt is asked to arrange it.

The plan itself is an ordinary field of state. The planner writes a list of steps, each with an `id`, a `goal` and the evidence that would settle it, beside a `revised` flag that records whether this plan replaced an earlier one. One detail of the implementation is worth reading, because only a running program produces it. `observations` accumulates and cannot be cleared, so immediately after a revision the last three observations are still the identical ones that triggered it, and both guards would fire again on the next turn. The run would replan and then die on the turn it recovered. The fix is a high-water mark, `replanned_at`, and both checks read `observations[replanned_at:]`.

Two runs. The recorded runs under `demo/runs/` are a pair worth reading against each other. In `v23-bc284c6b` the executor works for four turns, `submit` passes all three checks, and the validator accepts: twenty-five events, and the accepting decision is logged against the validator rather than the controller. In `v23-28e37a00` the mechanical checks fail twice, the run is routed back to the controller both times, and it ends in `error` after fifty-two events without the validator ever being consulted. That ordering is the design rule of [Section 4.7](#) visible in a trace: the cheap check runs first, and the judge is never asked about work that has already failed mechanically.

Neither recorded run replans. The edge exists in the compiled graph and no run under `demo/runs/` has taken it, so nothing here shows what a revision costs in practice.

DEMO STUB

Three artifacts are still missing. For in-run revision: a run in which `after_tools` routes to the planner, with the original plan, the observations that said nothing new, and the revised plan beside each other. For the validator: one candidate

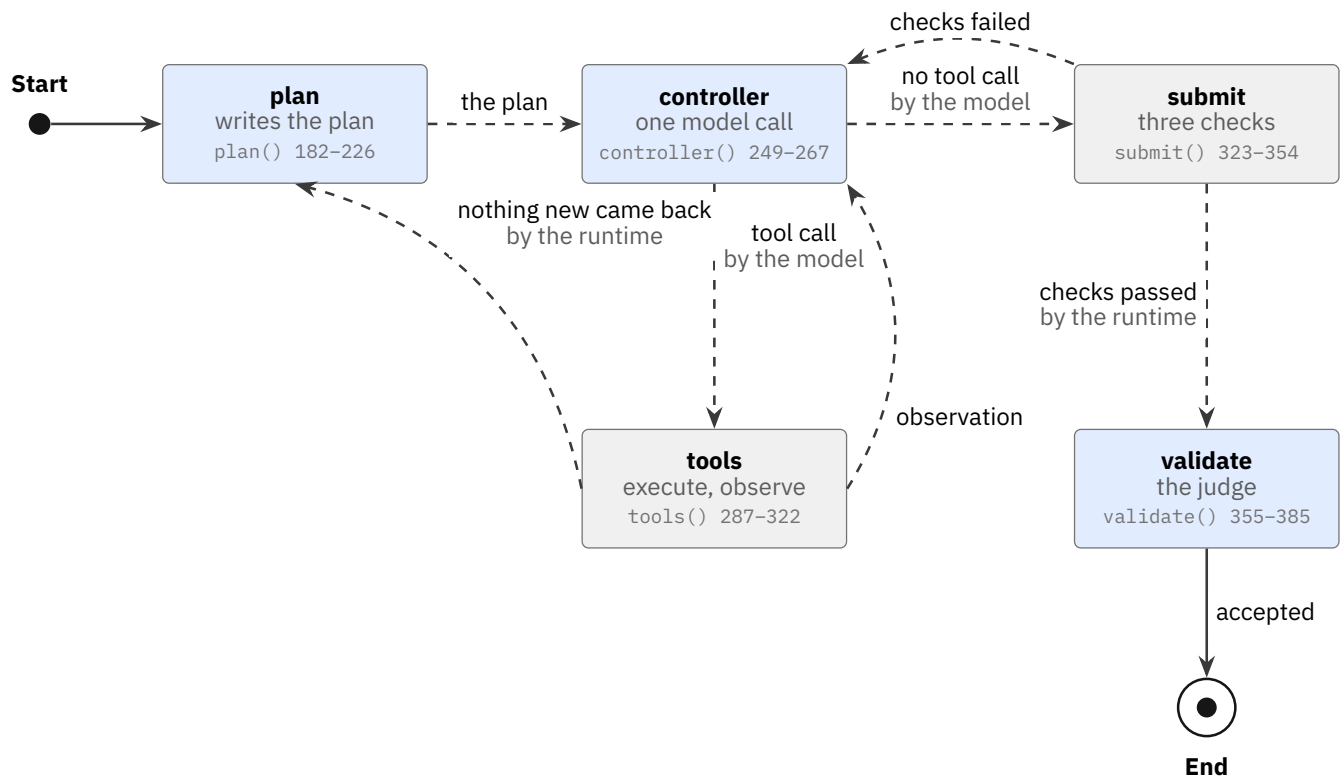


Figure 11 The DAG of the planning and validation demo, each node labelled with its method and line range in `graph.py`. Blue nodes are the three places where a model decides. The four routing predicates are `after_plan` (414–417), `after_controller` (404–413), `after_tools` (418–432) and `after_submit` (433–438). Compare [Figure 8](#): only `validate` reaches the exit.

that passes all three mechanical checks and is refused anyway, with the inputs it was given and the reasons it returned. For reflection: one refused attempt and the attempt that follows it, showing the entry appended to `reflections`, the fields the runtime cleared, and the first model input of the second attempt. `reflections` and attempt are named in `state.py` as the fields a later version brings, so the reflection artifacts need that version first.

4.11 Exercises

1. Take the four turns of the recorded loop run in [Figure 9](#) and write the plan that run would have needed for each turn to be checkable, one subgoal and one condition per turn. Say which of the four turns your conditions would have caught falling short, and which one they would have passed for the wrong reason.
2. The loop stops when the last three observations return the same result. Write a subgoal condition that catches a run this rule misses, and say what the condition reads that the rule does not.
3. A run replans twice and validates three candidates under a model-call budget of forty. Count the calls the planner and validator consume, say how many turns the executor has left, and name which limit fires first.
4. Give the validator authority to release a patch as well as refuse one. Using a false-positive rate of the order measured in [Section 4.7](#), say what the worst case costs on a batch of forty alerts, and why the mechanical checks are the cheaper place to keep the decision.
5. A rejected attempt leaves one entry in `reflections`. Write the entry you would want after a patch that cleared the scanner alert by renaming the flagged expression, and say which of the validator's three questions your entry would have to have been told about.
6. Argue the other side of the design rule in [Section 4.7](#): describe a fixture on which letting the validator release a patch is the right choice, and name the property of that fixture that ours does not have.

4.12 Reading

Before class: Sun et al. [17], §2 for the open-loop and closed-loop distinction, §§3.1 and 3.2 for plan generation and refinement, and Figures 1 and 2. Then Madaan et al. [12], §2 and Algorithm 1, for a refinement loop in which one model fills every role, and §3 of Shinn et al. [15] for all three of its models, the actor and evaluator split of [Section 4.5](#) and the self-reflection model and memory of [Section 4.8](#).

The counterweight is the part of the reading to do slowly. Huang et al. [6], §§1 and 3 with Tables 1 to 3, defines intrinsic self-correction and measures it. Stechly et al. [16], §5 with Tables 1 and 2, separates the verifier from the generator and varies only the verifier. Read both against the claim [Section 4.5](#) makes about who writes the value the exit edge reads.

For planning vocabulary beyond what this section uses, Russell and Norvig [14, §11.5.3] covers online planning, execution

monitoring, and replanning.

Looking ahead. The serial agent is now complete. A plan gives the run something to fall short of, the mechanical checks and the validator give it a refusal it cannot argue with, and reflection gives a refused attempt something to leave behind. Every version of it has been one model call at a time, on one trajectory, against one alert, and each addition was paid for in a guarantee we gave up. [Section 5](#) asks what changes when the work is divided among several agents instead, and what each division costs in the same currency.

5 Other agent design patterns

Learning objectives. After this section you will be able to name the design patterns in Google Cloud's guide [\[5\]](#) and sort them by one criterion: whether a model or predefined logic decides which agent runs next. You will be able to state, for the coordinator, hierarchical decomposition, parallel, and review-and-critique patterns, what changes in task ownership, state boundaries, model-call overhead, latency, failure propagation, and inspectability relative to the serial agent of [Section 4.8](#). And you will be able to answer the question that closes the block: given forty alerts, a fixed per-alert budget, and an evidence requirement, which of the five versions you would deploy, and which axis decides it.

5.1 The problem the reading addresses

The serial agent built in [Sections 2 to 4](#) is one point in a design space, and the space has names. The guide this section uses is supplementary engineering reading rather than study material: it is a vendor's architecture-center page, last updated 2026-05-28, whose purpose is to help a team choose an orchestration shape for a workload. It is cited here for two reasons. It names the patterns that appear in framework documentation and in job descriptions, so students will meet them. And its own classification of the patterns rests on the question this block has asked at every version: does a model orchestrate, or does predefined logic?

The guide opens with four requirements it says a team must settle before choosing. Does the task decompose into predefined workflow steps, or is it open-ended and in need of a model to orchestrate? Does the workload prioritize fast responses, or can it tolerate delay for accuracy? What is the inference budget, and does it support several model calls per request? Are decisions high-stakes enough to need a person in the loop? It adds a caution the course endorses: a predictable, structured workload that a single model call can serve does not need an agentic system at all. Those four questions are the syllabus's axes in a vendor's words, and [Table 10](#) returns to them.

5.2 Mechanism

The guide describes twelve patterns. [Table 8](#) lists them with the one property this course cares about first, the orchestration decision, and the tradeoff the guide itself states for each. Three of the multi-agent patterns (sequential, parallel, loop) are run by what the guide calls *workflow agents*, which operate on predefined logic without consulting a model for orchestration; the coordinator, hierarchical, and swarm patterns use a model to route or decompose. The review-and-critique and iterative refinement patterns are, in the guide's words, implementations of the loop pattern.

Four of these are the ones the lecture discusses, each with the question the teaching plan attaches to it.

Coordinator and specialist routing.

Who selects the worker, what context is passed to it, and who owns the returned result? In the guide's pattern the coordinator is a model call that analyzes the request and routes it; the guide contrasts this with the parallel pattern, whose dispatch is hardcoded.

Hierarchical task decomposition.

How are subgoals assigned, and how are partial results integrated? The guide presents this as the coordinator pattern applied recursively, for ambiguous problems that need research, planning, and synthesis.

Parallel work.

Which tasks are independent, and what does their eventual join require? The guide's example fans one input out to four specialists and gathers four outputs into one analysis.

Review and critique, and iterative refinement.

Where does feedback route control, and what causes the system to stop? The guide's critic can approve, reject, or return the output with feedback; its refinement loop stops at a quality threshold or an iteration cap.

The guide's own account of the ReAct pattern deserves one sentence, because it differs in emphasis from [Section 3](#): it describes the observation as something the model "saves in its memory", and it lists an error that prevents continuation as a terminating condition. Both are correct as descriptions of what a framework does; in this course's vocabulary the runtime records the observation and the runtime declares the terminal status, and [Section 3](#) said why that ownership matters.

5.3 Evidence

The guide reports no experiment, and none is claimed here. What it offers is a comparison that sorts the patterns by workload, reproduced in [Table 9](#). The sorting is useful because it makes the guide's implicit criterion visible: the first group is exactly the patterns whose orchestration is predefined, the second is exactly the patterns whose orchestration is a model, and the third is the patterns that loop. That is the design-time versus run-time question, and the termination question, applied to multi-agent systems. The academic papers of [Sections 2 to 4](#) remain the evidence for what each shape costs and buys; the guide is a map of the names.

Table 8 The guide's twelve patterns, sorted by who decides which agent runs next. Tradeoffs are the guide's own statements, paraphrased.

Pattern	Orchestrated by	What it is	Tradeoff the guide states
Single-agent	The model, within one agent	One model, one tool set, one system prompt; the model plans and picks tools	Degrades as tools and task complexity grow: latency, wrong tool choice, incomplete tasks
Sequential	Predefined logic	Specialized agents in a fixed linear order; each output is the next input	Lower latency and cost than model orchestration; rigid; cannot skip unneeded steps
Parallel	Predefined logic	Independent subagents run at once; a gather step synthesizes	Lower latency; higher immediate resource and token use; the gather step must reconcile conflicting results
Loop	Predefined logic	A sequence of subagents repeats until an exit condition (iteration cap or a state)	Risk of an unbounded loop if the exit condition is wrong or never reached
Review and critique	Predefined logic (a loop)	A generator produces; a critic evaluates against criteria and approves, rejects, or returns feedback	At least one extra model call per evaluation; latency and cost accumulate with each revision
Iterative refinement	Predefined logic (a loop)	One or more agents improve a result held in session state until a quality threshold or an iteration cap	Latency and cost per cycle; exit conditions must be designed
Coordinator	The model	A central agent decomposes the request and dispatches subtasks to specialists	More model calls than a single agent; higher token throughput, cost, and latency
Hierarchical task decomposition	The model, at several levels	A root agent decomposes; subagents decompose further until workers can execute	Considerable architectural complexity; harder to design, debug, and maintain; many model calls
Swarm	No central orchestrator; agents hand off to one another	All-to-all communication among specialists; a dispatcher starts the task; an explicit exit condition is required	The most complex and costly; risk of unproductive loops or failure to converge
Reason and act (ReAct)	The model, within one agent	Thought, action, observation repeated until an answer, an iteration cap, or an error	Higher end-to-end latency than one query; a misleading tool result can propagate to the answer
Human-in-the-loop	Predefined checkpoints	The agent pauses at a checkpoint and waits for a person to approve, correct, or supply input	Requires building and maintaining the external interaction system
Custom logic	Code the team writes	Conditional orchestration in code, mixing the other patterns	The team designs, implements, and debugs the whole flow

Table 9 The guide's comparison section, by workload type. The characteristics are the guide's, paraphrased.

Workload	Patterns the guide lists	Characteristics the guide gives
Deterministic	Sequential; parallel; iterative refinement	Steps known in advance; no model orchestration; a fixed sequence, or independent subtasks, or progressive improvement over cycles
Dynamic orchestration	Single agent; coordinator; hierarchical decomposition; swarm	The system plans, delegates, and coordinates without a script; higher latency and cost from the orchestrating model calls
Iteration	ReAct; loop; review and critique; iterative refinement	Cycles of refinement, feedback, and improvement; a distinct validation step; unpredictable latency while waiting for an exit condition
Special requirements	Human-in-the-loop; custom logic	Human supervision for high-stakes or subjective tasks; branching logic beyond a linear sequence

5.4 Which decisions the programmer made

Table 10 takes the four selected patterns and asks what each would change about the serial agent, axis by axis. The serial agent is the reference row: one process, one state record, one investigation at a time, model calls bounded by the attempt limit and the overall execution limit, failures contained within an attempt, and a trace that is one sequence.

Table 10 What each pattern changes relative to the serial agent of Section 4.8.

Axis	Coordinator	Hierarchical	Parallel	Review and critique
Task ownership	The coordinator owns the task and delegates pieces; the specialist owns only its piece	Ownership is nested; each level owns the decomposition it made	Each worker owns one independent piece; the gather step owns the whole	Unchanged: the generator owns the candidate, the critic owns acceptance
State boundaries	The coordinator's context and each specialist's are separate; what crosses is what the coordinator passes	A state boundary per level; partial results cross upward	One state record per worker; the join reads all of them	Unchanged: one record; the critic's reasons are added to it
Model-call overhead	One routing call per request plus the specialists' calls	Multiplied by the depth of the hierarchy	The sum of the workers' calls, run concurrently	One evaluator call per candidate, plus revisions
Latency	The routing call is on the critical path	Each level adds a round trip	Reduced to the slowest worker plus the join	Each revision adds a round trip
Failure propagation	A misrouted task fails in the wrong specialist; the coordinator may not notice	A wrong decomposition propagates down and its cost multiplies	Contained per worker until the join, where a missing piece blocks the whole	A critic that accepts a wrong candidate passes it on; one that rejects a right one costs a revision
Inspectability	The routing decision and its reasons are in the trace, if recorded	Traces nest; which level decided what must be recoverable	Traces interleave; which worker wrote what must be recoverable	Improved: acceptance carries reasons

Three things the table makes visible are worth stating. The two patterns that use a model to orchestrate (coordinator, hierarchical) move a control decision from design time to run time. Everything the course said about the loop applies to that decision: it is adaptive, its cost is not known in advance, and it must be recorded to be inspected. The parallel pattern changes no control decision at all; it changes who runs when, and its whole cost is at the join. That is Unit 2's subject, and HW2's. And the review-and-critique pattern is not new: it is the validator and the reflection step under another name. The guide's own tradeoff statement, that latency and cost accumulate with each revision, is the reason Section 4.8 put an attempt limit and an overall execution limit on the loop.

5.5 The demo

There is no sixth version. The demonstration for this section is to take the serial graph of Section 4.8 and say, for each selected pattern, which node would split and which edge would appear.

- **Coordinator.** The planner becomes a router. Instead of producing a plan for one executor, it selects among executors specialized by alert kind, and the edge from the planner fans out to whichever it chose. Task ownership passes to the chosen executor for the duration of its run and returns to the router with the candidate finding. The routing decision is a model call and must be traced as one.
- **Hierarchical decomposition.** The planner's subgoals stop being advisory entries in one state record and become dispatched tasks, each with its own executor and its own state. A new join node integrates the partial results before the validator sees a candidate. The graph gains a level, and so does the trace.
- **Parallel work.** Nothing inside the serial agent changes. Around it, forty alerts fan out to forty instances of the same graph, and a report node joins their findings. The join is where independence must be checked: two alerts that share a helper file are not independent in their evidence, only in their verdicts. This is HW1's report stage and, with bounded concurrency, HW2's experiment.
- **Review and critique; iterative refinement.** Already present. The validator is the critic, the reflection node and the reset are the refinement, and the two limits are the exit conditions. The one change the guide's version would make is to let the critic return feedback to the same generator without a reset; Section 4.8 said why this course resets the attempt and retains only the guidance.

FIGURE NEEDED: Google Cloud design-pattern diagrams

One SVG per pattern on the guide's page [5], content licensed CC BY 4.0. Credit each as "Google Cloud Architecture Center, *Choose a design pattern for your agentic AI system*, [pattern] diagram, CC BY 4.0." Every path below is under <https://docs.cloud.google.com/static/architecture/images/> and begins with choose-design-pattern-agentic-ai-system-.

Pattern	File name suffix
Coordinator	coordinator.svg
Hierarchical task decomposition	hierarchical-task.svg
Parallel	parallel.svg
Review and critique	review-critique.svg
Iterative refinement	iterative-refinement.svg
ReAct	react.svg
Loop	loop.svg

The block closes on the question that has the shape of a midterm question. Given forty alerts, a fixed per-alert budget, and a requirement that every verdict carry evidence references that resolve: which of the five versions would you deploy, and which of the four axes, adaptability, model-call overhead, failure propagation, inspectability, decides it? The answer is not in these notes; the argument for it is. The chain cannot follow evidence it did not anticipate, so it fails the evidence requirement on any alert whose helper delegates. The loop can, at a cost that is not known before the run, which the per-alert budget must cap. The validator is what makes the evidence requirement checkable rather than hoped for. Whether the reflection step earns its second attempt under a fixed budget is an empirical question the trace can answer, and it is the question HW1 asks teams that adopt it.

Part A ends here. After the break, Part B takes up the execution machinery that [Section 3](#) only pointed at: the state schema and its reducers, transitions and conditional routing as predicates over state, terminal states, tool signatures and the request, dispatch, observation, state-update cycle, and execution authority.

5.6 Exercises

- Using [Table 8](#), sort the twelve patterns into those where a model decides which agent runs next and those where predefined logic does. For each model-orchestrated pattern, name the trace event that would have to exist for the routing decision to be inspectable.
- The guide lists “iterative refinement” under both deterministic workflows and workflows that involve iteration ([Table 9](#)). Explain, using the distinction between decision timing and decision ownership from [Section 2](#), why a loop with a fixed exit condition can belong in both.
- Take the parallel row of [Table 10](#). Two of the forty alerts share one helper file, `helpers/separate_request.py`. Say what the join must check before it treats their findings as independent, and what the trace of each instance must record to make that check possible.
- Answer the closing question for a budget that permits, on average, one attempt per alert. Which version do you deploy, which axis decided it, and what single number from a trace would change your answer?

5.7 Reading

Supplementary, cited in lecture: Google Cloud Architecture Center [\[5\]](#), the requirements section, the pattern descriptions, and the comparison section. The academic papers of [Sections 2 to 4](#) and AIMA remain the principal study material for this block.

Part B: Execution machinery

6 The investigation record

The upshot

1. **Runtime state and model context are separate records whose contents overlap.** The runtime keeps the first between steps and assembles the second for each model call, and the fields that never enter a model call are what make accounting and control possible.
2. **A transition consumes a result that arrives from outside the program.** The next state is a function of the current state together with a model reply or a tool observation, so two runs from identical states can diverge without any node behaving differently.
3. **The reducer attached to a field fixes what every later step can see.** Appending retains a field's history, replacement installs the latest value and discards what it stood on, and the choice is made per field when the schema is written.
4. **A declared schema is not a runtime check.** Annotations describe the record for a type checker and validate nothing that enters it during a run, so every guarantee about the contents of state rests on code we write.

6.1 What connects one step to the next

A graph drawing shows where control is allowed to go, and it says nothing about what travels along the edges. What travels is one record, and the mechanisms of the two sections that follow, routing and dispatch, are both defined over it.

Definition 6.1 (State and the state schema). State is “a shared data structure that represents the current snapshot of your application” [8]. It is declared as a schema, a named set of fields with their types, which every node receives as input and updates as output.

We write one step of a run as two operations rather than one. A node reads the current state s_t together with a result e_t that arrived from outside the program, and it returns a partial update u_t . The runtime then merges that update into the record, key by key:

$$u_t = \text{node}(s_t, e_t), \quad s_{t+1} = \text{merge}(s_t, u_t).$$

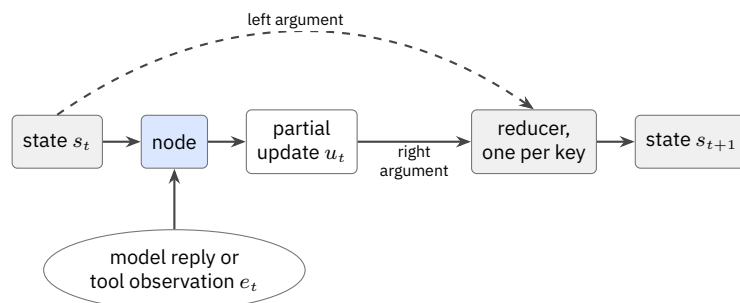


Figure 12 One transition. The node proposes an update; the runtime merges it. Each key is merged by its own reducer, whose left argument is the value already in state and whose right argument is the value the node returned.

The term e_t makes explicit what the current state does not determine. A model reply is sampled from a distribution, while a tool observation depends on a fixture that the run itself may have modified. Neither result is determined by s_t alone. This is the nondeterminism discussed in [Section 1.3: the same state can produce different successors](#). A run can therefore go wrong at this boundary even when every node behaves correctly.

A node returns only the keys it touched, because “the Node does not need to return the whole State schema - just an update” [8]. Our controller returns three keys out of the loop’s ten, messages, model_calls and usage, and it returns a single key, status, on the two paths where it stops the run before issuing a call.

The runtime, not the node, decides what merging means. It applies one reducer per updated key and saves the result, which is why a node can be written without knowing whether its contribution will be appended to a list or will overwrite what is there. Execution proceeds in super-steps, where “a super-step can be considered a single iteration over the graph nodes” [8], and the record is consistent at each super-step boundary rather than inside a node.

6.2 Model context, runtime state, durable state

Model context and runtime state are easy to confuse because much of their content overlaps. The model sees only what it needs for the current call; the runtime also tracks counters and other fields used to control execution. Saving enough of that state to resume after the process ends makes it durable.

The counters are the clearest case of a field that stays behind. Our loop refuses a model call once model_calls reaches forty and once the last three observations came back identical, and neither number is shown to the model. We keep them out on

Record	What it holds, and who reads it
Model context	The instructions, the selected history, and the observations supplied to one model invocation. Assembled per call, read by the model.
Runtime state	The record the runtime maintains between steps, including fields that enter no model call. Read by nodes, by routing functions, and by the limits we enforce.
Durable state	State written somewhere that survives the end of the process, in a form that supports resuming a run rather than restarting it.

Table 11 Three records, kept apart. Model context is derived from runtime state at each call; durable state is runtime state that has been written down.

purpose: a limit the model can read is a limit the model can argue with, and we want the check to be a comparison in the controller rather than an instruction in a prompt.

Now, durability. Kill the process in the middle of a run and start it again: what does the agent still know? For the loop as it stands, nothing. The record lives in memory, so a terminated process takes the whole investigation with it, and in-memory state is state all the same. The framework's answer is to compile the graph with a checkpointer, which gives "agents short-term memory" and "lets LangGraph applications keep useful information beyond a single graph run" [9].

Here, *durable* means only that the record can be read back after the process that wrote it is gone. It does not mean that resuming is free of consequences. Checkpoints are saved at super-step boundaries and not inside a node, so a run that resumes re-enters the interrupted node from its first line and repeats whatever that node had already done, including the calls it had already paid for. Persistence and recovery are taken up properly in the later units; what the record has to carry for this lecture is the run, not the restart.

6.3 The record is factored

How much structure a state record has is a design decision with consequences, and the ways of representing a state lie on an axis of increasing expressive power: atomic, factored, and structured [14, §2.4.7]. In an atomic representation each state is indivisible, a black box whose only property is being the same as or different from another. A factored representation splits each state into a fixed set of variables, each with a value. A structured representation describes objects, each with attributes of its own and relationships to other objects.

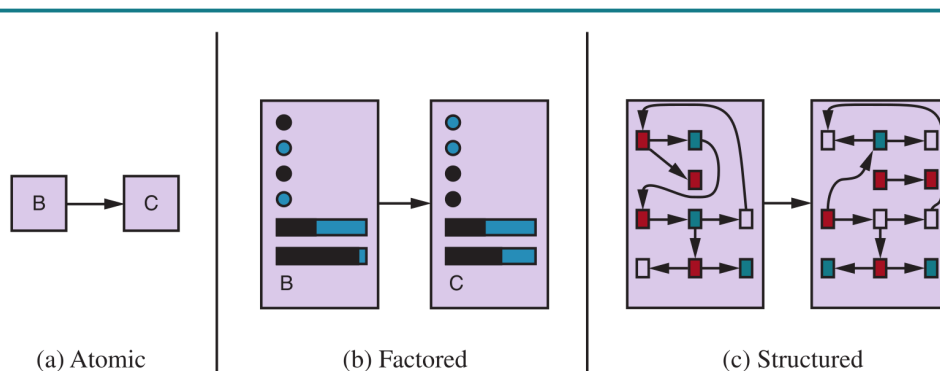


Figure 13 Three ways to represent states and the transitions between them. Figure 2.16 of Russell and Norvig [14], reproduced from the Global Edition (© Pearson Education Limited 2022).

mini-swe-agent uses all three, in different places.

- **Atomic.** The outcome of the run. Accepted or stopped is a label with no internal structure, and two runs that end in the same label are indistinguishable at that level. The number a benchmark reports is a count of such labels.
- **Factored.** The record itself. The alert, the observations, the candidate patch, the counters and the status are named fields with values, and two states can agree on some and differ on others.
- **Structured.** The patch and the rejection. A patch is a set of hunks, each over a file and a range of lines; a rejected submission names the check that failed, the file, and the line.

Keeping the record factored is what buys us the machinery of the rest of Part B: one reducer per named field (Section 6.4), routing predicates that read a field rather than parse a sentence (Section 7), and limits that are comparisons on a counter. Structure is confined to the two values where the model has to defend its work, and **the expressive representation is the exception rather than the default**, because reasoning over structure is harder for us and for the model both [14, §2.4.7].

Table 12 is the schema as `state.py` declares it. Its fields have three lifetimes: i) identity, set once, ii) run-local, carried across the whole run, and iii) attempt-local, cleared when the reflection step starts a new attempt.

Nothing in that declaration checks anything. A TypedDict is a description for a type checker, not a gate: its types cannot be used in `isinstance` tests, and an instance "returns an ordinary dictionary object at runtime" [10]. A model reply that puts a string where the schema declares an integer produces a state that type-checks in the editor and is wrong at run time, and the check that catches it is the argument validation of Section 8, which we wrote.

Field	Lifetime	Reducer	What it holds
alert, repo, commit, workspace	identity	none	The scanner alert under investigation, the fixture, the pinned commit, and the copy of the tree this run may touch.
messages	run-local	append	The conversation, in provider form.
observations	attempt-local	append	One record per dispatched tool call, whether it succeeded or was refused.
candidate_patch	attempt-local	replace	The patch currently proposed.
plan	attempt-local	replace	The subgoals and their status, written by the planner.
validation	attempt-local	replace	The latest acceptance decision and its reasons.
reflections	run-local	append	Guidance retained from attempts that were rejected.
attempt	run-local	explicit	Which attempt is running, under the runtime's control.
model_calls, usage	run-local	accumulate	Calls issued and tokens spent, kept across attempt resets.
status	run-local	replace	Running, or the terminal status that ended the run.

Table 12 The state schema of `state.py`. The loop populates the identity fields, messages, observations, candidate_patch, the counters and status; each later version adds the field its new node fills, so a version's diff shows one change rather than a field that was always waiting.

6.4 Reducers, and what a later step is allowed to see

A reducer is a small function with a large consequence: it decides whether a field accumulates a history or keeps only its latest value, and therefore what evidence is available to every step that comes after.

Definition 6.2 (Reducer). “Each key in the State has its own independent reducer function” [8]. Every reducer is a binary function of two positional arguments, the value already stored in state for that key and the update returned by a node, and where none is specified the default applies: it “ignores the left argument and replaces the state value with the right argument” [8].

We use three, and each one answers a question about visibility rather than about storage.

- **Append.** observations is annotated with `operator.add` and messages with `add_messages`, so every tool result of the current attempt stays readable. The controller's stall check needs exactly this: it compares the last three observations and stops the run when they are identical, which is a question no replaced field could answer.
- **Replace.** candidate_patch, plan, validation and status take the default. Each names what is true now, and a superseded plan is not evidence of anything except that the run changed its mind.
- **Accumulate.** usage is merged by `add_usage`, which sums four token counters and produces no total. Each node reports only the cost of its own call, so no node ever holds the running total and no node can overwrite it. The four counters stay apart because they are not interchangeable: a cache read costs a fraction of fresh input, and a single number cannot answer whether cache reads rose while fresh input fell.

The reset between attempts is where these choices are felt. When the validator rejects a candidate and the reflection step writes guidance for the next try, the new attempt has to begin with the attempt-local fields cleared and the run-local ones intact. Returning an empty list for observations does not do it: **an append reducer given an empty update leaves every earlier observation exactly where it was**, so the next attempt would inherit the reasoning it was supposed to be freed from. Clearing a field with an append reducer takes an explicit assignment under the runtime's control, and the runtime records it in the trace as a state change. Identity, reflections, the counters and the usage ledger survive the reset by design, because an attempt limit enforced on a counter that resets with the attempt is not a limit at all.

6.5 The demo: one tool result becoming state

Four events into `runs/v1-bc284c6b.jsonl`, the model has asked to read the file the alert names, and the dispatcher has returned 1,665 characters of source. Figure 14 shows the record before the merge, the update the tools node returns, and the model input assembled from the result.

Three things in that transition are worth holding on to. The node returned two keys and the loop's record has ten, so the counters advanced only where a node said so. The observation the runtime stored is richer than the message the model receives, because it keeps the arguments and the success flag alongside the text, and routing and accounting read those fields rather than the prose. And the model input for the next call is built from messages alone: `model_calls` stood at one throughout and was never shown, which is precisely how a budget stays a runtime control.

The trace is a fourth record, and it is the one that outlives the process. Every transition worth auditing is written to it as an event, `tool_request` and `tool_result` from the dispatcher, `model_call` from the controller, `routing` from the conditional edges, `state_change` from an attempt reset, and `terminal` once, at the end. State answers what the run knows now; the trace answers what the run did, and Section 7 rests on the second.

```
# 1. state before the merge (the keys this transition touches)
{"messages": [system, user, ai(tool_call read_file)], "observations": [],
 "model_calls": 1, "usage": {"input": 914, "output": 49, ...}, "status": "running"}

# 2. the update the tools node returns, and the merged result
return {"observations": [{"tool": "read_file",
    "args": {"path": "testcode/BenchmarkTest00283.py"},
    "ok": True, "result": "<1665 chars of source>"}],
 "messages": [ToolMessage(content=..., tool_call_id=...)]}
# merged: observations grows by one (operator.add), messages grows by one
# (add_messages), and every other key is untouched because it was not returned

# 3. the next model input, assembled from state["messages"] alone
[system, user, ai(tool_call read_file), tool(result)]
```

Figure 14 One tool result entering the record, from runs/v1-bc284c6b.jsonl, steps 3 and 4. The node names two keys; the runtime merges each one by its own reducer.

6.6 Exercises

1. Take the schema of [Table 12](#) and mark, for each field, whether the model ever sees its value. For every field you mark “no”, say which decision in the runtime reads it.
2. add_usage sums four counters and produces no total. Write the one question about a run that a single total makes unanswerable, and name the pair of counters whose movement answers it.
3. A colleague clears observations between attempts by returning an empty list for that key from the reflection node. Say what the record contains after the update is merged, and give the smallest change that produces the intended reset.
4. Our controller refuses a model call once forty calls have been issued. Suppose instead the limit were stated in the system prompt and the counter were removed from state. Name one run in which the two designs give different outcomes, and say which record you would inspect to tell them apart.

6.7 Reading

Before class: LangChain [8], the sections on state, schemas, reducers and super-steps, which are the implementation reference for everything in this section. §2.4.7 of Russell and Norvig [14] supplies the atomic, factored and structured distinction and Figure 2.16.

For reference while writing HW1: Lehtosalo [10] on what a TypedDict does and does not check at run time, and LangChain [9] on checkpoints, threads and stores, which is the machinery behind the durability row of [Table 11](#).

Looking ahead. The record is now the thing every later mechanism reads. The next section takes the code that reads it to choose the next node, and choosing to stop is one of those choices. We work through conditional edges, the two back edges that return a run to the planner, and three endings that a trace can establish: a run that completed, a run that declared a stop, and a run whose trace records no ending at all.

7 Routing, feedback loops, and termination

The upshot

1. **A routing decision is executable code over state.** A model proposes values, the runtime records them, and a routing function applies the transition rules, so a router that reads the model’s prose has handed the control decision back to the model.
2. **A back edge needs an exit predicate and a bound.** An acceptance condition states when a loop may stop and does not establish that it will, so what terminates a loop is a limit the runtime owns and checks.
3. **A dynamic path does not require a dynamic graph.** The compiled graph is fixed for the run, and different values in state select different paths through it.
4. **Termination is a predicate the runtime evaluates and records.** A trace establishes a completed assessment, a declared stop, or no observed ending, and the third of these is not an error until other evidence says so.

7.1 Routing is code, not prose

A conditional edge is where a run’s next step is chosen, and the choice is a comparison written in Python rather than a sentence written by a model. Both things are true at once, which is what makes the arrangement worth describing carefully: the model’s output decides which way the comparison comes out, and the comparison decides where the run goes.

Definition 7.1 (Conditional edge). A conditional edge is a function that receives the current state and returns the name of the next node, or the terminal node END, which is “a special node that represents a terminal node” [8]. The function runs in the runtime’s process and its return value is the transition.

Three responsibilities sit on three different components, and keeping them apart is the whole of this section.

- **The model proposes.** It emits a tool call, a plan, a candidate finding, or a judgement about whether a candidate is acceptable. Every one of those is a value, not an action.

- **The runtime records.** It merges the proposal into state through the reducers of [Section 6.4](#), and it writes what happened to the trace.
- **The router decides.** It reads named fields and returns a node name, applying the transition rules and the limits we fixed when we wrote it.

Our loop's router is `after_controller`, and what it reads is the point. It branches on `status`, and then on whether the last message carried tool calls; there is no third condition. It never reads the model's prose, so a model that writes "I am confident the patch is correct, submit it" moves the run exactly as far as a model that writes nothing: the branch is taken by the presence or absence of a tool call, not by the sentence. `after_submit` is one line, returning `END` when the submission was accepted and the controller otherwise.

The trace records who decided, and this is the field to read when a run surprises us. A routing event carries `by="model"` when the model's output selected the branch and `by="runtime"` when a limit did. **Neither value means the model chose the edge**, because the edge was chosen when we wrote the predicate; `by="model"` names the origin of the value the predicate tested.

7.2 The same graph, different paths

Compilation fixes the graph and nothing else. `compile` adds three nodes, two conditional edge sets and one unconditional edge, and the result is the same object on every run. What differs between two runs on the same alert is the sequence of states, and therefore the sequence of edges that the routers select.

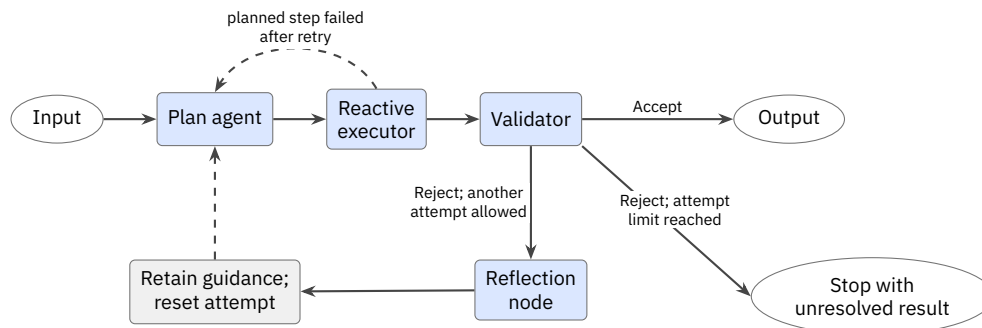


Figure 15 The complete serial graph, with both back edges. The upper edge revises a plan inside an attempt; the lower path runs the reflection step and begins a new attempt. Every edge in the drawing is a line of wiring code, and every label on one is a predicate over state.

The unconditional edge is the one that made the loop a loop: the return from `tools` to `controller`. Delete that line and the same three nodes become a chain. The conditional edges are what give one compiled graph its different shapes at run time, and the run in [Figure 9](#) traverses the controller four times without the graph changing once.

7.3 Two returns to the planner

[Figure 15](#) has two back edges, and they answer different questions. One asks what to do about a step that did not work; the other asks what to do about an attempt that did not work. Collapsing them produces a system that either rewrites its plan over every transient tool error or retries a whole investigation over one failed edit.

Back edge	Trigger	Retained	Reset
In-run plan revision	A planned step is still unsuccessful after the runtime's one free retry. The planner receives the failed subgoal and the error.	The attempt's observations, its progress so far, and the accounting.	The remaining subgoals, which the planner rewrites.
New attempt after reflection	The validator rejects the candidate and the attempt limit permits another try.	Reflections, identity, the counters and the usage ledger.	Observations, plan, candidate patch and validation, all set explicitly.

Table 13 The two back edges of [Figure 15](#). The retry that precedes the upper edge costs no model call, which is what makes it the runtime's decision rather than the model's.

Ownership divides along the same line. The free retry is ours: the runtime reissues a failed tool call once without consulting anyone, because a transient failure is not evidence about the plan. The revision that follows a second failure is the model's proposal, recorded by the runtime and merged into plan. The new attempt is ours again, because it is a comparison on the attempt counter, and **a limit that the model can talk its way past is not a limit**.

7.4 What actually stops a loop

Every loop in [Figure 15](#) has an exit predicate, and no predicate is a guarantee. The validator's acceptance condition says when an attempt may end; it says nothing about whether any attempt will satisfy it, and a run whose candidate is rejected every time will keep going until something other than acceptance stops it. Three limits do that work, and all three are comparisons the runtime performs on fields it owns.

- **The model-call budget.** Forty calls per run, and the controller checks it before it issues a call rather than after. A limit checked afterwards is a post-mortem: the call has already been paid for. Exceeding it sets `budget_exhausted` and the router sends the run to `END`.
- **The stall check.** Three consecutive observations with identical results end the run with `no_progress`. Comparing results rather than requests is deliberate, because it also catches six distinct searches that all return nothing, which is the repeated-action failure mode that ReAct reports for its own trajectories [21, §3.3].
- **The attempt limit.** Two attempts, counted in a field that survives the reflection reset. An attempt counter that reset with the attempt would bound nothing.

The framework contributes one more, underneath all of these: a recursion limit on the number of super-steps a single execution may run, which raises an error when it is reached [8]. It is a failsafe against a wiring mistake and not a control on the investigation, because it knows nothing about alerts, attempts or costs. We prefer our limits to fire first, and the budget of forty is set low enough that they do.

7.5 What a trace establishes

A run ends in one of five ways, and the vocabulary is closed on purpose: `accepted`, `unresolved`, `budget_exhausted`, `no_progress` and `error`. The trace writer refuses a status outside that set, and it also refuses the opposite mistake. Closing a trace that never recorded a terminal event raises `NoTerminalStatus`, because **a run that never said how it ended is a defect in our instrumentation rather than an outcome we can report**.

That is what we control. What a trace lets a reader conclude is a separate question, and it has three answers rather than five.

What the trace records	What it supports
A verdict, then a terminal event with status <code>accepted</code>	A completed assessment under the runtime's acceptance rules, whose quality is a separate question from its completion.
A terminal event with a stop status and its reason	A declared end to execution without a completed assessment, with whatever partial evidence the run had gathered.
No terminal event	Completion has not been established by this record. The run may still be active, may have been interrupted, or may have written an incomplete trace.

Table 14 Three readings of a trace. The third is a statement about the evidence and not about the run, and treating it as `error` attributes a cause the record does not carry.

Three distinctions keep those readings honest. Errors and outcomes are not the same thing: a refused tool call is an observation inside a run that can still end in `accepted`, and a process killed between two events leaves a run with no status at all. The finding and the execution status are not the same thing either, because an assessment that reports insufficient information is a completed assessment, while a declared stop can carry partial evidence and no finding. And reaching `END` establishes only that the graph stopped: the reason lives in status, which we set.

Termination therefore stays where routing is. A model's statement that it is finished is a proposal, like every other value it produces; the controller applies the completion rules, sets the status, and the trace records what was set. Stopping conditions and token-budget accounting are taken up in their own right in the coming weeks.

7.6 The demo: three endings

The recorded run of Figure 9 ends the first way. Its seventeenth event is a `terminal` record with status `accepted` and the run's usage totals, written after `submit` reported that all three mechanical checks passed, and the routing event just before it carries `by="runtime"`. Nothing in that trace is a model's opinion about whether the work was done.

A budget-exhausted run ends the second way, and its shape differs in one instructive place. The controller's last event is not a model call, because the check that fired precedes the call: the trace shows a `routing` event with decision `budget_exhausted` and `by="runtime"`, then the terminal event, and no fifth model call between them. The evidence the run had gathered is still in observations, and the absence of a finding is not the absence of work.

The third ending is the one we cannot label. Interrupt a run between its tool result and its next model call and the trace stops there, with events that are all individually valid and no `terminal` record at the end. What we may say about that file is that no ending was observed. What we may not say is which ending would have arrived, and an excerpt used in teaching is worth marking with how it was produced.

7.7 Exercises

1. `after_controller` reads two fields. Write a model reply that would make a reader expect the run to end and that in fact routes to tools, and say which field decided.
2. The budget check sits before the model call. Move it after, and describe the two ways the resulting trace would differ from the current one on a run that exhausts the budget.
3. A colleague proposes ending a run when the model writes `DONE` in its message. Give one run in which that predicate ends the run too early and one in which it never fires, and name the field each failure would have read instead.
4. Take a trace whose last event is a tool result. List everything you can state about how that run ended, and mark each statement with the event that supports it.

7.8 Reading

Before class: LangChain [8], the sections on conditional edges, END and the recursion limit. §3.3 of Yao et al. [21] supplies the repeated-action failure mode that the stall check is built against, with its own trajectory counts.

For reference while writing HW1: the terminal statuses and trace events of `trace.py`, which are the vocabulary every claim in Table 14 is made in.

Looking ahead. Routing decides which node runs next, and one of those nodes executes what the model asked for. The last section of these notes takes that step apart. We ask what a tool contract contains beyond the schema the model sees, what the dispatcher does between a request and a result, which artifact proves that a function was entered, and which component refuses a request that the run is not allowed to make.

8 The tool boundary

The upshot

1. **A tool contract spans the model-facing definition and the runtime's enforcement.** The provider's schema carries the name, the description and the argument types; the result type, the effect class and the execution constraints live in code we write and are enforced there.
2. **A model response establishes that a tool was requested, not that one ran.** The evidence that a function was entered is the runtime's own record of entering it.
3. **A refused request and a permitted call that fails are different events with the same shape.** Both become observations the model can read, and only one of them reached the implementation.
4. **Interface design changes outcomes with the model held fixed.** The action set, the feedback format and the guardrails have measurable consequences, which puts them in the architecture rather than in the plumbing.

8.1 The tool contract

A tool has two faces, and only one of them is visible to the model. The model reads a name, a description and an argument schema; the runtime holds the function, the constraints on where it may operate, and the decision about what happens when an argument is wrong. Calling the first face the tool is the mistake this section is built to prevent.

Definition 8.1 (Tool contract). We use *tool contract* for the complete specification of one tool. It has five parts: i) the name and purpose, ii) the typed arguments and their constraints, iii) the typed success result and the error representation, iv) the side-effect class, and v) the execution constraints. The side-effect class is read-only inspection of source for two of our three tools, and the execution constraint on all three is the directory they may touch. The model-facing definition covers the first two parts. The other three are properties of the implementation and its dispatcher, and a model that has read the description knows nothing about them.

Our three tools make the contract concrete. `read_file` takes a path and an optional line range and returns text; `search` takes a pattern and an optional glob and returns matching lines with their locations; `edit` replaces an exact string in a file. All three take the workspace root as their first parameter and resolve every path through it, and none of them can write outside that root. The signature is the contract, which decides where configuration goes. Tools read the run's configuration from the module rather than accepting it as a parameter, because the dispatcher validates the model's arguments by binding them against the signature, so **every parameter in a tool's signature is a value the model is permitted to supply**. A `config` parameter would be bindable, and a bindable parameter is one the model can set. The workspace root is a parameter, and it is the dispatcher that supplies it; the model never sees it in the schema and cannot pass it.

8.2 The dispatch cycle

Between a model's request and a result there are five operations, and each of them can end the cycle. Figure 16 follows one request through all of them.

- **Resolve the name.** A request naming a tool that is not in the registry is refused, and the refusal names the tools that do exist. The registry is built from the tool enumeration, so there is one list and no way for a tool to be advertised to the model without being registered.
- **Bind the arguments.** The dispatcher binds the model's arguments against the function's signature before calling it, which turns a wrong argument name into a readable refusal instead of a `TypeError` raised from somewhere inside the tool. Binding checks names and arity and not types, so unambiguous type mismatches are coerced rather than spending a model call to complain about a string where an integer was wanted.
- **Check the policy.** Every path argument is resolved inside the workspace root, and a path that resolves outside it is refused before any file is opened.
- **Execute.** The function runs with the validated arguments and the dispatcher's workspace root.
- **Observe.** Success and failure alike become an `Observation` carrying the tool, the arguments, an ok flag and a string result, and the dispatcher writes a `tool_result` event either way.

A refusal is an observation and not an exception, and the difference is what keeps the loop of Section 3 alive. An exception ends the run; an observation goes back to the model, which reads what went wrong and chooses again. When `edit` refuses

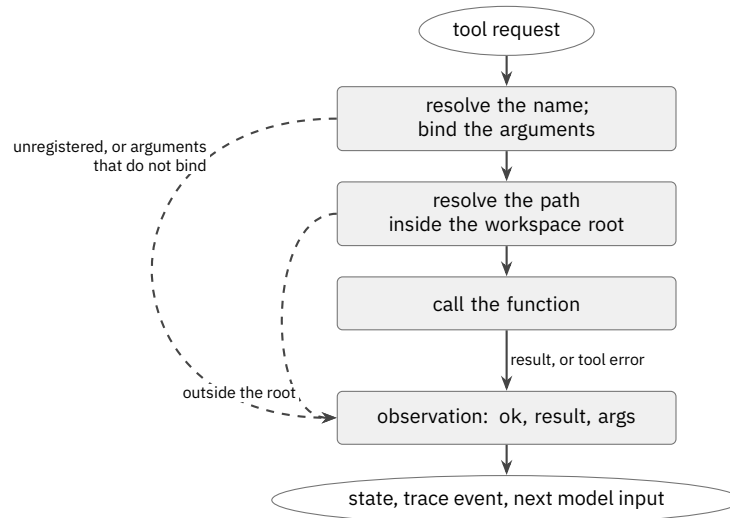


Figure 16 One dispatch. The name is resolved against the registry and the arguments are bound against the signature; the path is resolved inside the workspace root; the function runs. Every path in the drawing produces an observation, which is what allows a run to continue after a request the runtime would not carry out.

an ambiguous target the model retries with more context, and a run that raised instead would die on its first imprecise edit. Exceptions stay for the genuinely unrecoverable, such as a trace that cannot be written.

8.3 What the model saw, and what ran

The same tool call leaves traces in six places, and they support different claims. Table 15 follows the `read_file` call of the recorded run through all of them.

Artifact	What it contains	What it establishes
Model input	The tool definitions and the conversation so far.	Which operations were offered, and on what description.
Model response	A tool call with an identifier, a name and arguments.	That the model requested an operation. Nothing more.
<code>tool_request</code> event	The tool name and the arguments as the dispatcher received them.	That the request reached the dispatcher and the cycle began.
Dispatcher	Registry lookup, argument binding, path resolution.	That the request satisfied the contract, or the reason it did not.
<code>tool_result</code> event	<code>ok=true</code> , 1,665 characters.	That the function ran and returned, and how much text came back.
State and next input	One record appended to observations, one tool message appended to messages.	What the next model call was allowed to see.

Table 15 One `read_file` call on `testcode/BenchmarkTest00283.py` through the artifacts it leaves, from `runs/v1-bc284c6b.jsonl`, steps 3 and 4. The provider’s raw request stays in the message history; the trace records the dispatcher’s view of it.

The second and fifth rows are the pair worth separating. A model response containing a tool call is evidence about the model and not about the file system, and a transcript that shows a request followed by a plausible-looking result establishes nothing on its own, because both were produced by the model. The runtime’s `tool_result` event is what establishes that the function was entered, and it is our record rather than the model’s.

The demonstration is sequential on purpose. When a model returns several tool calls in one response, the dispatcher runs them one at a time and returns one observation for each, including the ones that failed, because a tool call left without an answer makes the next model message malformed.

8.4 The interface is a design decision

The action set and the shape of the feedback are not conveniences around the real system; they move outcomes. SWE-agent is the clearest evidence for that, because it holds the model fixed and varies the interface.

Definition 8.2 (Agent-computer interface). An agent-computer interface is “an abstraction layer between the LM agent and computer” [20], which “specifies both the commands available to the LM and how the environment state is communicated back to the LM” [20, §2].

Four principles come out of that work, and each has a counterpart in our dispatcher. Actions are simple and few; important operations are consolidated so that one action makes real progress; feedback is informative and concise; and guardrails catch

the mistakes models make, so that an error does not propagate through the turns that follow [20, §2]. With GPT-4 Turbo, the resulting system resolves 12.47% of the 2,294 SWE-bench test instances, against 3.8% for the previous best non-interactive retrieval-augmented system, and the same interface carries to another model, with Claude 3 Opus resolving 10.5% [20, §1]. The ablations of the interface bear on design decisions rather than on leaderboards (Table 16).

Component	Variant	Resolved	Change
SWE-agent, as configured		18.0	
Editor	edit action without linting	15.0	−3.0
	no edit action at all	10.3	−7.7
Search	iterative search	12.0	−6.0
	no search tools	15.7	−2.3
File viewer	30-line window	14.3	−3.7
	whole file	12.7	−5.3
Context	full history	15.0	−3.0
	no demonstration	16.3	−1.7

Table 16 Percentage of SWE-bench Lite instances resolved under ablations of the interface, with the base model held fixed. Table 3 of Yang et al. [20].

Two readings of that table are worth separating. It supports the claim that interface choices move outcomes by several points each, and the largest gap, 7.7 points between having an edit action and not having one, is larger than the gap between many published systems. It does not support a general ranking of interfaces, because every row varies one component of one system, evaluated on a 300-instance subset with a single base model; a variant that helps this agent with this model on these instances may not transfer. The row worth dwelling on is the search comparison, where an interface that returns results one at a time scores below having no search tool at all, 12.0 against 15.7: a well-intentioned interface can be worse than nothing when it invites the model to work through every match in turn.

8.5 Execution authority

Everything in Part B converges on one sentence, and it is worth stating plainly: the model proposes, the runtime executes, and the runtime may refuse. What makes that more than a slogan is that each clause names a component we can point at, and each component sits on the path the request has to travel.

Two different distinctions hide inside the word “control”, and they come apart under pressure.

- **Decision timing and ownership.** Some choices are fixed when we write the program and others are computed while it runs, which is the question of Section 1.4. Timing does not tell us who owns a choice: a routing function that runs at run time is still code we wrote, and a limit evaluated mid-run is still our limit.
- **Proposal and execution.** A requested action, a claimed result, or a declaration that the work is finished becomes effective only by passing through the runtime’s execution path. A model that writes that it has already run the tests has produced a sentence.

Guarantee	Where it is enforced
Only registered tools run	The registry built from the tool enumeration, consulted before anything is called.
Arguments satisfy the contract	Signature binding and coercion in the dispatcher, before the call.
No file outside the workspace is read or written	<code>workspace.resolve</code> , called by every tool on every path argument.
Source inspection does not modify the fixture	The tool implementations, which open files for reading, and the pinned commit the diff is taken against.
An exhausted budget prevents another model call	The controller’s check, made before the call is issued.
A back edge obeys the attempt limit	The routing function and the attempt counter of Section 7.3.

Table 17 Each guarantee of the demo, and the component that enforces it. No row is enforced by a sentence in a prompt.

`workspace.resolve` is where an ordinary-looking shortcut fails. It resolves both the root and the candidate path fully before comparing them, so that `..` segments collapse and symlinks are followed and neither can be used to step outside. Comparing the two resolved paths as strings would look equivalent and would not be, because `/tmp/ws` is a prefix of the unrelated directory `/tmp/ws-evil`. An absolute path in the request lands outside the root when it is joined to it, and is refused for that reason rather than honoured.

8.6 The demo: a refused request

Waiting for a model to produce a forbidden request is a poor way to demonstrate a boundary, so we construct one. Two requests go to the same dispatcher, one after the other: a permitted `read_file` on the file the alert names, and a `read_file` whose path climbs out of the workspace. Only the arguments differ.

- **The request.** `{“name”: “read_file”, “args”: {“path”: “../../etc/passwd”}}`, presented as a constructed request rather than one the model made.

- **The check that refuses it.** `workspace.resolve` raises `PathEscapeException`, the dispatcher catches it, and no file is opened.
- **The record.** An observation with `ok=false` whose result is the refusal message, and a `tool_result` event carrying the same reason.
- **The evidence that nothing ran.** The trace holds a `tool_request` event and a failed `tool_result` event with no successful result between them, and the instrumented implementation records no entry.

Set beside the permitted request, the pair shows the branch: identical tool, identical dispatcher, and the argument decides whether execution happens. That is the whole of the ownership question, and it is answered in code rather than in a prompt.

8.7 Exercises

1. Take the six artifacts of [Table 15](#) and mark the smallest set that would convince you a file was read. Say what each of the others would still allow to be true.
2. A tool is added whose signature is `def rescan(ws, path, config)`. Name what the model gains that it did not have before, and give the change that removes it.
3. In [Table 16](#), an iterative search interface scores below having no search tool. Name the agent behaviour reported behind that result, and say which of the four design principles it violates.
4. Our dispatcher refuses a path outside the workspace root. Write the request you would send to test that the check is on the execution path rather than in the prompt, and say which trace event would tell you the check was skipped.

8.8 Reading

Before class: §§2–3 and Figure 3 of Yang et al. [20], for the interface design principles and what the agent actually sees, and Table 3 for the ablations of [Table 16](#).

For reference while writing HW1: Anthropic [3] for the exact request and result representation a provider emits and expects, and `dispatch.py` and `workspace.py` for the checks that stand between the two.

Looking ahead. Part A asked which decisions we fix at design time, and Part B has given the machinery that carries them: a record with one reducer per field, routers and limits that read that record, and a dispatcher that decides what a request is allowed to do. HW1 uses the same fixture and the same tools for triage rather than repair. The question it puts to you is the one these notes have been working toward: for every decision your agent makes, name the component that made it and the artifact that proves it.

References

- [1] Talor Abramovich, Meet Udeshi, Minghao Shao, Kilian Lieret, Haoran Xi, Kimberly Milner, Sofija Jancheska, John Yang, Carlos E. Jimenez, Farshad Khorrami, Prashanth Krishnamurthy, Brendan Dolan-Gavitt, Muhammad Shafique, Karthik Narasimhan, Ramesh Karri, and Ofir Press. EnIGMA: Interactive tools substantially assist LM agents in finding security vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, 2025.
- [2] Mohamed Aghzal, Gregory J. Stein, and Ziyu Yao. Why do LLM-based web agents fail? a hierarchical planning perspective. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. Table and figure numbers follow arXiv:2603.14248v2, 27 April 2026.
- [3] Anthropic. Tool use with claude. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>, 2026. Accessed 2026-09-18. HW1 reference material, not assigned reading.
- [4] Paul Gauthier. Aider: AI pair programming in your terminal. <https://aider.chat>, 2024. Source of the Search/Replace edit format, as cited by Xia et al.
- [5] Google Cloud Architecture Center. Choose a design pattern for your agentic AI system. <https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system>, 2026. Page last updated 2026-05-28; accessed 2026-09-16. Content licensed CC BY 4.0.
- [6] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] LangChain. LangGraph graph API. <https://docs.langchain.com/oss/python/langgraph/graph-api>, 2026. HW1 setup material, not assigned reading.
- [9] LangChain. Persistence. <https://docs.langchain.com/oss/python/langgraph/persistence>, 2026. Accessed 2026-09-18. HW1 reference material, not assigned reading.
- [10] Jukka Lehtosalo. PEP 589: TypedDict, type hints for dictionaries with a fixed set of keys. <https://peps.python.org/pep-0589/>, 2019. Accessed 2026-09-18. Final; incorporated into the typing module.
- [11] Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. An empirical study on failures in automated issue solving, 2025. arXiv:2509.13941v1, 17 September 2025.
- [12] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. Figure and table numbers follow the NeurIPS proceedings PDF.
- [13] OWASP Benchmark Project. BenchmarkPython. <https://github.com/OWASP-Benchmark/BenchmarkPython>, 2026. Pinned at commit f1291485808b66e20ddb6b01b10dc71b3df8c8ba (2,540 files, 1,230 test cases; verified 2026-09-15). Answer keys and bundled scan results are stripped from the copy the agent sees.
- [14] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education Limited, Harlow, 4th, global edition, 2022. Global Edition of the fourth edition (Pearson, 2021). Quotations, figures, and page numbers in these notes are from this printing, in which Chapter 2, Intelligent Agents, occupies pp. 54–79. Section and figure numbers agree with the United States printing.
- [15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. Byline as on the NeurIPS proceedings title page (five authors; the arXiv v4 source lists six). Figure and table numbers follow the proceedings PDF.
- [16] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *International Conference on Learning Representations (ICLR)*, 2025.
- [17] Haotian Sun, Yuchen Zhuang, Linghai Kong, Bo Dai, and Chao Zhang. AdaPlanner: Adaptive planning from feedback with language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. Figure and table numbers follow the NeurIPS proceedings PDF.
- [18] Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents, 2026. arXiv:2601.22311v1, 29 January 2026; section, table and figure numbers follow v1.
- [19] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754. Article FSE037. Section and figure numbers in these notes follow the FSE camera-ready, not the arXiv preprint (arXiv:2407.01489), which carries the “Agentless” title prefix and a different section numbering.
- [20] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. Numbers read from the NeurIPS camera-ready: §1 and Table 3.
- [21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. ICLR version, arXiv:2210.03629v3.